

第2章: TypeScript の基礎

この章では、TypeScript（タイプスクリプト：JavaScript に「型」の仕組みを追加したプログラミング言語。読みは「タイプスクリプト」）の基本を学びます。TypeScript は JavaScript（ジャバスクリプト：ブラウザで動く代表的なプログラミング言語）に「型（Type：データの種類のこと。例えば「文字」「数値」「真偽値」など）」という仕組みを追加した言語です。「型って何？」という方も安心してください。身近な例えを交えながら、一つずつ丁寧に解説します。

この章で学ぶこと

- **型（Type：タイプ）とは何か** — データの種類を指定する仕組み。「この箱には数字だけ入れていいよ」というラベルのようなもの
- **基本的な型** — `string`（ストリング：文字列を表す型）、`number`（ナンバー：数値を表す型）、`boolean`（ブーリアン：true/false の真偽値を表す型）など
- **インターフェース / 型エイリアス** — 自分だけのオリジナルの型を定義する方法。書籍管理アプリでは「Book型」を作ります
- **ジェネリクス（Generics：ジェネリクス。「総称型」と訳される。型をパラメータ（後で決める変数のようなもの）として受け取る仕組み）** — 型をパラメータ（引数）として受け取る柔軟な仕組み
- **よくあるエラーと対処法** — 初心者がつまづきやすいTypeScriptのエラーメッセージと、その読み方

なぜTypeScriptを学ぶの？ JavaScriptだけでもアプリは作れますが、TypeScriptを使うと「コードを書いている途中で間違いに気づける」「エディタ（コードを書くソフト）が賢く補完してくれる」というメリットがあります。最初は少し面倒に感じるかもしれませんが、慣れると「TypeScriptなしでは開発できない！」と思うようになります。

大事なポイント（先に伝えておくと安心）：

- **型は実行時には消える** — TypeScript の型情報は「人間とTypeScript コンパイラだけが見えるメモ」。プログラムが実際に動くときには影響しません（これを「型消去」と呼びます）。
- **`const` をデフォルトにする** — 値を入れ替える必要がないなら必ず `const`（再代入禁止）。「ここは入れ替わらない」とコードを読む人に伝えられて、バグも減ります。
- **`null` と `undefined`** — どちらも「値がない」を表しますが、`undefined`（アンディファインド：未定義）は「まだ何も入れていない」、`null`（ヌル：明示的な空）は「意図的に空にしている」を表します。
- **セミコロン `;` は省略可能** — 文末の `;` は本当は無くても動きますが、本書では分かりやすさのため付けます。

目次

0. 前提知識: JavaScriptの基礎の基礎
1. TypeScript とは
2. 基本的な型
3. 型注釈と型推論

4. インターフェースと型エイリアス
5. ジェネリクス
6. ユニオン型とリテラル型
7. TypeScript の設定 (tsconfig.json)
8. よくあるエラーと対処法
9. 発展: アプリでは使っていない重要なTypeScript機能

0. 前提知識: JavaScriptの基礎の基礎

TypeScript は JavaScript に「型」を追加した言語なので、JavaScript の文法をひと通り知っておくと話が早く進みます。ここでは TypeScript を読むうえで最低限必要な JavaScript の文法だけ、ぎゅっと圧縮して解説します。「もう知ってる!」という方は読み飛ばして 1 章へ。

動かしながら読みたい方へ: 以下の例はどれも、ターミナルで `node` と入力して Enter を押すと開く対話モード (REPL) に貼り付けて実行できます。終了は `Ctrl+C` を2回押します。または、ブラウザの**開発者ツール** → **Console** (F12 で開ける) にそのまま貼り付けてもOKです。

0.1 コメント

コメントは「コンピュータには無視されるが、人間が読むためのメモ」です。

```
// この行はコメント。実行されない。           // 行頭の // からその行の終わりまでが
「単一行コメント」。コンピュータは無視するので、人間用のメモとして自由に書ける。
let x = 1; // 行末にも書ける                     // let = 後から値を変えられる変数の
宣言。x = 1 で変数 x に 1 を代入。文末の ; はセミコロン (文の終わりを示す)。// 以降はコメント。
                                                    // ↑ 空行も自由に入れられる (コー
ドの読みやすさのため)。
/*                                                    // /* で始めると複数行コメントの
開始。                                                    // * は飾り (書式の慣習)。意味はな
* 複数行コメント。                                                    // 文章中の /* や */ も「ただの文
い。                                                    // */ で複数行コメントの終了。
* /* と */ で囲む
字」として扱われる。
*/
```

0.2 変数の宣言: `const` / `let`

値に名前を付けて保存する仕組みです。本書では原則 `const` (コンスト: constant の略。再代入できない変数) を使い、必要なときだけ `let` (レット: 再代入できる変数) を使います。古い書き方の `var` (バー: variable の略。古い変数宣言) はバグの元なので使いません。

なぜ **const** をデフォルトにするのか？ 変数を見ただけで「これは中身が変わらない」と分かったら、コードを読む人の負担が減ります。「もしかして他で書き換えてる？」と疑う必要がないからです。React/Next.js のコードではほぼ全ての変数を **const** で宣言します。

▼ このコードがやること（先に日本語で）：値に名前を付けて保存する「変数」を、**const**（あとで変えられない）と **let**（あとで変えられる）の2通りで作ってみます。一番大事なのは「**変えない値は const を使う**」という習慣——うっかり書き換えてしまうバグを未然に防げます。詳しい1行ごとの意味はコード内のコメントを見てください。

```
const name = "太郎"; // const = 一度入れたら変えられない // const は「定数」宣言。
name という名前の箱を作り、"太郎" を入れる。="代入演算子"。"... " で囲うと文字列リテラル。
// name = "次郎"; // ❌ エラー: Assignment to constant variable. // ↑ コメントアウト
// している。もし // を外すと「定数 name に再代入してる」とエラーになる。

let count = 0; // let = 後で書き換え可能 // let は「後で値を変
えられる変数」の宣言。count という変数に初期値 0 を入れる。
count = count + 1; // ✅ OK // count に「現在
の count + 1」を代入。count は 0 → 1 になる。
console.log(count); // console.log
は「コンソール（ターミナルやブラウザの出力欄）に値を表示する」関数。count の中身 (=1) を表示する。

// ▼ 実行結果
// 1
```

console.log() とは？: ターミナル（Node.js の場合）またはブラウザのコンソール（ブラウザの場合）に値を表示する関数です。プログラムの動作確認に多用します。

0.3 基本のデータ型

データ型	例	説明
文字列 (string)	"hello" '太郎' `テンプレ \${x}`	クォートで囲んだ文字
数値 (number)	42 3.14 -0.5	整数も小数も同じ型
真偽値 (boolean)	true false	ハイorロー
null / undefined	null undefined	値が「ない」ことを表す (null=明示的、undefined=未定義)
配列 (array)	[1, 2, 3]	値の並び
オブジェクト (object)	{ name: "太郎", age: 20 }	キー：値の組

▼このコードがやること（先に日本語で）：プログラムでよく使う基本的な「データの種類（型）」——文字列・数値・真偽値・配列・オブジェクト——を1つずつ変数に入れて、`console.log` でまとめて表示します。今は「こういう種類のデータがあるんだな」と眺めるだけでOKです。それぞれの値の書き方はコメントで説明しています。

```
const text = "hello"; // 文字列リテラル "hello" ("..." または
'...' で囲ったもの) を const 変数 text に入れる。
const num = 42; // 数値リテラル 42 (クォートで囲わない数字)
// 変数 num に入れる。整数も小数も同じ number 型。
const isOk = true; // 真偽値 true (true か false の2つしかない)
// を変数 isOk に入れる。命名で is... を付けると「真偽値」と分かりやすい慣習。
const list = [1, 2, 3]; // 配列リテラル。[ ] の中に値を , で並べ
// る。[0]=1, [1]=2, [2]=3 の順に並ぶ (添え字は 0 始まり)。
const user = { name: "太郎", age: 20 }; // オブジェクトリテラル。{ キー: 値 } の組を
// , で並べる。 .name や .age で値を取り出せる。

console.log(text, num, isOk, list, user); // console.log は引数を " , " 区切りでま
// けて表示する。複数値の確認に便利。

// ▼ 実行結果
// hello 42 true [ 1, 2, 3 ] { name: '太郎', age: 20 }
```

0.4 演算子（足し算・比較）

```
console.log(1 + 2);           // 3（足し算）           // + は加算演算子。数値同
士なら算術加算、文字列同士なら連結。
console.log(10 - 3);          // 7                     // - は減算演算子。10
から 3 を引く。
console.log(4 * 5);           // 20                    // * は乗算演算子（×
の代わり）。
console.log(10 / 3);          // 3.3333333333333335     // / は除算演算子（÷
の代わり）。JavaScript の数値は浮動小数点なので小数になる。
console.log(10 % 3);          // 1（余り）              // % は剰余演算子。10 ÷
3 = 3 余り 1 の「余り」を返す。
console.log("ab" + "cd");     // "abcd"（文字列の連結） // + の左右が文字列だと、
足し算ではなく連結になる。

console.log(1 === 1);         // true（等しい。型もチェック） // === は「型も値も完全に
同じか」を判定。同じなら true、違うなら false。
console.log(1 === "1");       // false（型が違っていると false） // 数値の 1 と文字列の
"1" は型が違うので false。
console.log(1 !== 2);         // true（異なる）          // !== は「型または値が
異なるか」を判定。違うなら true。
console.log(3 > 2);           // true                    // > は「左が右より大
きいか」。等号付きなら >=（以上）。
console.log(true && false);    // false（両方trueならtrue = AND） // && は AND 演算子。
両辺が true のときだけ true。
console.log(true || false);   // true（片方trueならtrue = OR）  // || は OR 演算子。ど
ちらか片方が true なら true。
console.log(!true);           // false（反転）           // ! は否定演算子。true
→ false、false → true に反転する。
```

`==` ではなく `===` を使う: 1個少ない `==` は型変換しながら比較するため、`1 == "1"` が `true` になります。バグの温床なので、本書では一貫して `===` を使います。

0.5 文字列とテンプレートリテラル

▼このコードがやること（先に日本語で）：文字列に変数の値を埋め込む2つの方法——古い「`+` でつなげる」やり方と、新しい「テンプレートリテラル（バッククォート ``` で囲み `${変数}` を埋め込む）」やり方——を見比べます。覚えてほしいのは後者の ``~${name}~`` の書き方で、こちらの方が読みやすくミスも減ります。

```

const name = "太郎"; // 文字列リテラルを変数 name に代入。

// 連結（古い書き方）
const msg1 = "こんにちは、" + name + "さん!"; // + 演算子で「文字列+変数+文字列」をつ
なげる。冗長になりがち。

// テンプレートリテラル（新しい書き方。バッククォート ` で囲む）
const msg2 = `こんにちは、${name}さん!`; // `...` で囲んだ中に ${変数} を書くと、
その場所に値が埋め込まれる。読みやすい。

console.log(msg1); // 1行目の文字列を表示。
console.log(msg2); // 2行目の文字列を表示。

// ▼ 実行結果
// こんにちは、 太郎さん！
// こんにちは、 太郎さん！

```

`${ ... }` の中には変数や式（しき：値を計算する記述。`1 + 2` や `name.length` など、評価すると1つの値になるもの）を入れられます。テンプレートリテラル（template literal：バッククォートで囲んだ文字列）は改行も含められて便利です。

0.6 if文（条件分岐）

▼ このコードがやること（先に日本語で）：点数に応じて「優・可・不可」を出し分ける条件分岐を `if / else` で書きます。「もし〜なら、こうする。そうでなければ...」という考え方で、上から順に条件をチェックし、最初に当てはまったブロックだけが実行されます。

```

const score = 75; // 変数 score に数値 75 を入れる。

if (score >= 80) { // if は「もし〜なら」の意味。( ) の中の条件
  console.log("優"); // score が 80 以上のとき表示。
} else if (score >= 60) { // else if = 「上の条件に当てはまらず、こちら
  console.log("可"); // 60 以上 80 未満なら表示。
} else { // else = 「どの条件にも当てはまらないな
  console.log("不可"); // 60 未満なら表示。
} // if 文の終わり。

// ▼ 実行結果
// 可

```

`{ ... }` で囲んだ範囲を**ブロック**（block：複数の文をひとまとめにした区切り。中括弧で囲む）と呼びます。条件に当てはまったブロックだけ実行されます。

「文（statement）」と「式（expression）」の違い: `if (...) { ... }` のように「処理を1つ実行する命令」が文。`1 + 2` や `score >= 80` のように「評価すると1つの値になる」のが式。`const x = 1 + 2;` は「`x` に `1+2`（式）を代入する文」のように、文の中に式が入ります。

0.7 for ループ・配列のループ

▼このコードがやること（先に日本語で）：同じ処理を繰り返す2つの書き方を学びます。①回数を数える `for` ループ ②配列の中身を1つずつ取り出す `for...of` ループです。配列を扱うときは ②の `for...of` が読みやすくおすすめです。

```
// 0, 1, 2 と3回繰り返す
for (let i = 0; i < 3; i++) {           // for ループ。( 初期化; 継続条件; 各回の最後の処理 ) の3つを ; で区切って書く。let i = 0 で開始、i < 3 が true の間繰り返し、毎回終わりに i++ (i = i+1) 。
  console.log(`i = ${i}`);             // ループの中身。テンプレートリテラルで現在の i を表示。
}                                       // for ブロックの終わり。

// ▼ 実行結果
// i = 0
// i = 1
// i = 2

// 配列を1つずつ取り出す（モダンな書き方）
const fruits = ["apple", "banana", "cherry"]; // 文字列の配列を作る。
for (const fruit of fruits) {                 // for...of ループ。fruits の中身を 1つずつ fruit に取り出して繰り返す。const なので毎回新しい束縛。
  console.log(fruit);                         // 取り出した値を表示。
}                                             // for ブロックの終わり。

// ▼ 実行結果
// apple
// banana
// cherry
```

0.8 関数の書き方

関数は「処理に名前を付けて何度でも呼び出せるようにしたもの」です。3通りの書き方があります。本書では主に3番の**arrow関数**を使います。

▼このコードがやること（先に日本語で）：「2つの数を足す」という同じ処理を、関数の4通りの書き方で作って結果を見比べます。注目してほしいのは `(a, b) => a + b` というアロー関数——React/Next.js でほぼ毎回登場する書き方なので、ここで形に慣れておきましょう。

```
// 1. function 宣言
function add1(a, b) {
  (a, b) が「仮引数（受け取る値）」
  return a + b;
  undefined を返す。
}

// 2. function 式
const add2 = function (a, b) {
  return a + b;
};

// 3. アロー関数 (=> を使う、Reactでよく使う)
const add3 = (a, b) => {
  return a + b;
};

// 3'. アロー関数の省略形 (return 1行のとき)
const add4 = (a, b) => a + b;

console.log(add1(1, 2)); // 3
console.log(add2(1, 2)); // 3
console.log(add3(1, 2)); // 3
console.log(add4(1, 2)); // 3
```

// function キーワードで関数を定義。add1 が関数名、
// return = 計算結果を呼び出し元に返す。これがないと
// 関数本体の終わり。
// 「無名関数」を変数 add2 に代入する書き方。function
// の後に関数名がない。
// 中身は宣言版と同じ。
// 式の終わりなので最後に ; が付く（宣言版は不要）。
// (引数) => { 処理 } の形がアロー関数。function キーワードが不要で短い。
// 本体は通常関数と同じ。
// アロー関数も「式」なので末尾に ;。
// 本体が return 1行だけのときは { } と return を省略可能。「a + b の値が自動で返る」と読む。
// 関数の呼び出し。add1 に 1 と 2 を渡し、返回值 3 を表示。
// どの書き方でも結果は同じ。

アロー関数の魅力: `function` の文字を書かなくて済むので短く、コールバック関数（あとで呼んでもらう関数）として渡すときに見やすくなります。React/Next.jsのコードはほぼアロー関数で書かれています。

0.9 配列の便利メソッド: map / filter / forEach

これは超重要です。後の章で頻繁に登場します。

▼このコードがやること（先に日本語で）：配列を扱う3つの超重要メソッド—— **forEach**（1つずつ処理）、**map**（1つずつ加工して新しい配列を作る）、**filter**（条件に合う要素だけ残す）——を使い分けます。鉄則は「**map** と **filter** は元の配列を変えず、新しい配列を返す」こと。React で一覧表示を作るとき毎回使うので、ここでしっかり押さえましょう。

```
const numbers = [1, 2, 3, 4, 5]; // 数値の配列を用意。

// forEach: 1つずつ処理する
numbers.forEach((n) => { // forEach は「配列の各要素に対して関数を
  console.log(n); // n には 1 → 2 → 3 → 4 → 5 と順に入
  // } // } // } // } // }
}); // } // } // } // } // }
// (undefined) // }

// ▼ 実行結果
// 1
// 2
// 3
// 4
// 5

// map: 1つずつ加工して新しい配列を作る
const doubled = numbers.map((n) => n * 2); // map は「各要素を加工した新しい配列を返
console.log(doubled); // 加工後の配列を表示。

// ▼ 実行結果
// [ 2, 4, 6, 8, 10 ]

// filter: 条件を満たす要素だけ残した新しい配列を作る
const evens = numbers.filter((n) => n % 2 === 0); // filter は「条件式が true になる要素だ
console.log(evens); // 偶数だけ残った配列を表示。

// ▼ 実行結果
// [ 2, 4 ]
```

0.10 オブジェクトの読み書き

▼このコードがやること（先に日本語で）：「名前・年齢...」のような複数の情報をまとめて持つオブジェクトの、値の取り出し・書き換え・追加、そして最後に「**分割代入**（**const { name, age } = user;** で一気に取り出す）」を学びます。特に分割代入は React のコードで頻出するので、形を覚えておくと後がラクです。

```

const user = { name: "太郎", age: 20 }; // オブジェクトを作る。name と age の2つの
プロパティ（プロパティ=オブジェクトのキーと値の組）を持つ。

// 値の取り出し（2つの書き方）
console.log(user.name); // "太郎"（ドット記法） // .（ドット）でプロパティ名を直接指定する
書き方。普通はこちらを使う。
console.log(user["name"]); // "太郎"（ブラケット記法） // [ ] にキー名を文字列で渡す書き方。変
数でキーを指定したいとき便利（user[キー名変数]）。

// 値の書き換え
user.age = 21; // const で宣言されていても、オブジェ
クトの「中のプロパティ」は書き換え可能（const が禁止するのは user 自体の差し替えのみ）。
console.log(user.age); // 21 // age が 21 に変わった。

// プロパティの追加
user.email = "taro@example.com"; // 存在しないキーに代入すると、新しいプ
ロパティが追加される。
console.log(user); // user 全体を表示すると、email が追
加されているのが分かる。
// ▼ 実行結果
// { name: '太郎', age: 21, email: 'taro@example.com' }

// 分割代入（オブジェクトから一気に取り出す）
const { name, age } = user; // { } の中に取り出したいキー名を書く
と、同じ名前の変数を一気に作れる。name = user.name, age = user.age と同じ意味。
console.log(name, age); // "太郎" 21

```

分割代入のコツ: React のコードでよく `const { title, author } = book;` のような書き方を見ます。これは「`book` オブジェクトから `title` と `author` を取り出して同名の変数を作る」省略記法です。慣れると非常に短く書けます。

0.11 セミコロン ; と改行

JavaScript/TypeScript では文末にセミコロン ; を付ける習慣があります。実は省略しても大体動きますが（自動セミコロン挿入 = Automatic Semicolon Insertion, ASI という仕組みがあるため）、本書では付ける派です（VS Code が自動で付けてくれる場合も多い）。

省略すると稀にバグる例: `return` のすぐ後で改行すると、自動で ; が入って `return undefined;` と解釈されることがあります。曖昧さを避けるため、本書では一貫して ; を付けます。

```
const a = 1;           // const で a に 1 を入れる。文末の ; は「ここで1つの文が  
終わる」を示す。  
const b = 2;           // 同様に b に 2 を入れる。  
console.log(a + b);    // a + b は 3 になる。それを表示。
```

これで JavaScript の超基礎は OK。次の節からいよいよ TypeScript 本編に入ります。

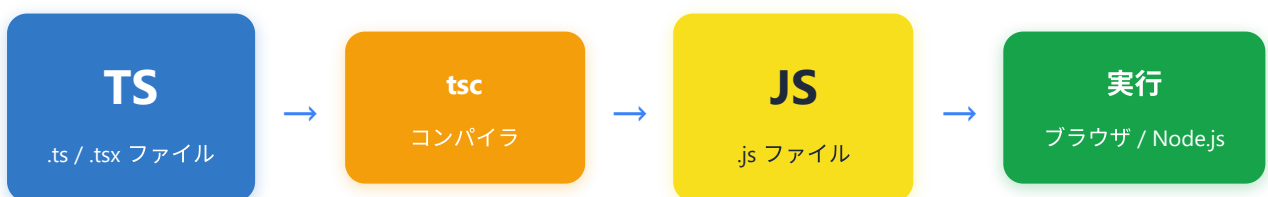
1. TypeScript とは

1.1 JavaScript との関係

TypeScript は、Microsoft（マイクロソフト：Windows や VS Code を作っている会社）が開発した **JavaScript のスーパーセット（superset：上位互換。JavaScriptの機能を全て含み、さらに追加機能がある言語）** です。すべての JavaScript コードは有効な TypeScript コードですが、TypeScript には「**型システム**」（Type System：タイプシステム。変数や関数に入るデータの種別を事前に定義し、チェックする仕組み）という強力な機能が追加されています。

TypeScript のコードは直接ブラウザや Node.js（ノードジェイエス：パソコン上で JavaScript を動かす実行環境）で実行できません。必ず **コンパイル**（Compile：コンパイル。人間が書いたプログラムを、コンピュータが実行できる形に変換すること。TypeScriptの場合はJavaScriptに変換する処理を指す。「トランスパイル（transpile）」とも呼ばれる）という変換処理を経て、JavaScript に変換されてから実行されます。

身近な例え： TypeScriptとJavaScriptの関係は、「下書き用紙」と「清書用紙」に似ています。TypeScript（下書き）には赤ペンで注意書き（型情報）が書いてあり、間違いがあればその場で気づけます。最終的にコンパイルすると、注意書きを消した綺麗なJavaScript（清書）ができあがります。



この流れをもう少し詳しく見てみましょう。

開発時

開発者が TypeScript でコードを書く

TypeScript コンパイラ (tsc) が型チェックを実行

型エラーがある場合 ▼

コンパイルエラー
開発者に通知

型エラーがない場合 ▼

JavaScript に変換

← 修正して再度書く

実行時

ブラウザまたは Node.js で実行

ポイント（とても重要）：TypeScript の型情報は、コンパイル後の JavaScript には一切残りません。型はあくまで「開発時の安全ネット」です。これを **型消去（Type Erasure：タイプレイジャー。型情報がコンパイルで削除される性質）** と呼びます。

例えば `const age: number = 25;` は、コンパイル後には `const age = 25;` になります。型注釈の `: number` の部分が消えるイメージです。つまり「実行時に型でエラーを止める」ことはできません。型チェックはコーディング中の VS Code と、コンパイル時の `tsc` だけが担当します。

1.2 なぜ TypeScript を使うのか

TypeScript を使う最大の理由は **型安全性** です。以下の表で、JavaScript と TypeScript の違いを比較してみましょう。

観点	JavaScript	TypeScript
型チェック	実行時にエラー発覚	コンパイル時にエラー発覚
エディタ補完	限定的	強力な自動補完
リファクタリング	手動で確認が必要	型の変更で影響範囲を自動検出
ドキュメント性	コメントに頼る	型そのものがドキュメント
バグ発見のタイミング	ユーザーが使用中に発覚	開発中に発覚
学習コスト	低い	やや高い（型の学習が必要）
大規模開発	困難	適している

1.3 JavaScript vs TypeScript の比較コード例

例1: 関数の引数に誤った型を渡すケース

▼ このコードがやること（先に日本語で）：同じ「挨拶を作る関数」を JavaScript 版と TypeScript 版で書き比べ、「間違った型の値を渡したとき何が起きるか」を見ます。JavaScript は誤りに気づかず動いてしまうのに対し、TypeScript は実行する前にエラーで教えてくれる——これが TypeScript を使う一番のメリットです。

```
// ---- JavaScript ----
function greet(name) {
  型は書けない（書こうとすると構文エラー）。
  return "こんにちは、" + name + "さん！";
  何の型でも文字列に変換されて連結されてしまう。
}

// 数値を渡してもエラーにならない（実行時まで気づけない）
console.log(greet(42));
JavaScript は何も警告しない。
// => "こんにちは、42さん！" ← 意図しない動作
// 42 が文字列 "42" に勝手に変換されて連結される。バグの温床。

// function 宣言。引数 name の
// 文字列連結で挨拶を作る。name が
// 引数に数値 42 を渡しても、
// 42 が文字列 "42" に勝手に変換さ
```

```
// ---- TypeScript ----
function greet(name: string): string { // : string は「型注釈 (Type Annotation)」。name の右の : string は「引数 name は string 型」、) の右の : string は「戻り値も string 型」を表す。
    return "こんにちは、" + name + "さん!"; // ロジック自体は JavaScript と同じ。
}

// コンパイル時にエラーが発生!
console.log(greet(42)); // ↑ ここで TypeScript が「string が必要なのに number を渡している」と警告。実行する前に気づける。
// エラー: Argument of type 'number' is not assignable to parameter of type 'string'
// (number 型の引数は、string 型のパラメータに代入できません)

// 正しい使い方
console.log(greet("太郎")); // 文字列を渡すと OK。
// => "こんにちは、太郎さん!"
```

例2: オブジェクトのプロパティアクセス

▼ このコードがやること（先に日本語で）: オブジェクトのプロパティ名を**タイプミス**したときの挙動を、JavaScript と TypeScript で比べます。JavaScript は黙って **undefined** を返すだけ（バグに気づきにくい）ですが、TypeScript は「そんなプロパティは無いよ」と赤線で即座に指摘し、正しい名前まで提案してくれます。

```
// ---- JavaScript ----
const user = { // user オブジェクトを作る。
    name: "田中", // name プロパティに文字列。
    age: 25, // age プロパティに数値。 , (カンマ) でプロパティを区切る。最後のカンマはあっても無くてもよい (trailing comma)。
};

// タイプミスしてもエラーにならない (実行時に undefined になる)
console.log(user.nmae); // => undefined ← バグ! // name と nmae (タイプミス)。JavaScript は存在しないプロパティを許し、結果は undefined。表示してから「あれ?」と気づくことが多い。
```

```
// ---- TypeScript ----
const user = { // 型注釈を書かなくても、TypeScript が自
動で { name: string; age: number; } と「推論」してくれる。
  name: "田中",
  age: 25,
};

// タイプミスするとコンパイル時にエラー！
console.log(user.nmae); // ↑ VS Code が即座に赤線。
// エラー: Property 'nmae' does not exist on type '{ name: string; age: number; }'.
// ('nmae' というプロパティは { name; age } 型には存在しません)
// Did you mean 'name'? // 「もしかして name のこと？」とサジェス
トもしてくれる。

// 正しいアクセス
console.log(user.name); // => "田中"
```

例3: 配列操作での型安全性

▼ このコードがやること（先に日本語で）：「数値だけの配列」のつもりが、うっかり文字列を混ぜてしまったときの違いを比べます。JavaScript では計算結果が **"15six"** のように壊れてしまいますが、TypeScript では **number[]**（数値だけの配列）と宣言しておくことで、文字列を入れた瞬間にエラーで止めてくれます。

```
// ---- JavaScript ----
const numbers = [1, 2, 3, 4, 5]; // 配列リテラル。JavaScript では配
列の中身の型は問わない。

// 文字列を追加してもエラーにならない
numbers.push("six"); // ← バグの原因になる // push は配列末尾に要素を追加するメソ
ッド。文字列でも入ってしまう。

// 後で計算しようとするすると予期しない結果に
const sum = numbers.reduce((a, b) => a + b, 0); // reduce は配列を左から畳み込んで
1つの値にするメソッド。第2引数 0 は初期値。a が累積値、b が現在の要素。
console.log(sum); // => "15six" ← 文字列連結になってしまう！ // 数値 + 文字列 → 文字列連結の
ルールが発動。"1+2+3+4+5=15" + "six" = "15six"。
```

```
// ---- TypeScript ----
const numbers: number[] = [1, 2, 3, 4, 5]; // number[] は「number 型の配列」。これで「数値以外は入れられない」と TypeScript が保証してくれる。

// 文字列を追加しようとするとコンパイルエラー！
numbers.push("six"); // ↑ 「number しか入らない配列」に string を push しようとして即エラー。
// エラー: Argument of type 'string' is not assignable to parameter of type 'number'

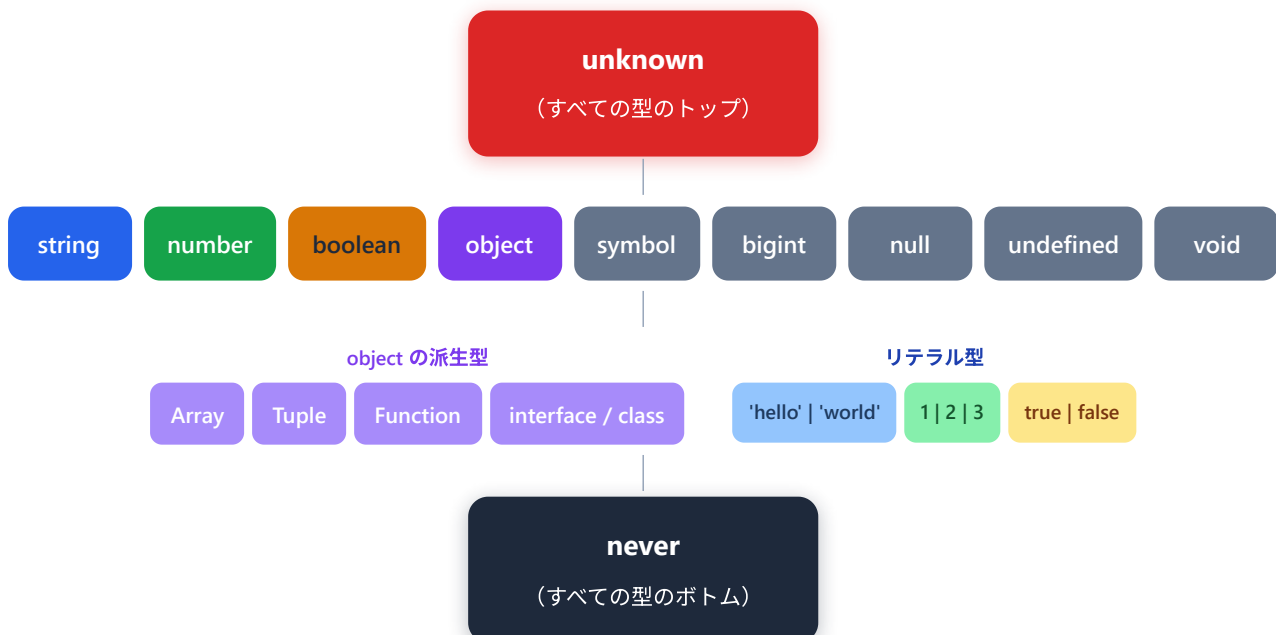
// 正しい使い方
numbers.push(6); // 6 は number なので OK。
const sum = numbers.reduce((a, b) => a + b, 0); // a, b ともに number と推論されるので、+ は算術加算で動く。
console.log(sum); // => 21 // 1+2+3+4+5+6 = 21
```

2. 基本的な型

TypeScript には豊富な型が用意されています。ここでは、すべての基本型を詳しく見ていきます。

TypeScript の型階層

まず、TypeScript の型システム全体を俯瞰してみましょう。



ポイント: **unknown** はすべての型の「親」で、**never** はすべての型の「子」です。**unknown** にはどんな値も代入できますが、**never** にはどんな値も代入できません。

2.1 string（文字列型）

文字列を表す型です。シングルクォート、ダブルクォート、テンプレートリテラルのいずれでも使えます。

▼ このコードがやること（先に日本語で）：「文字列」を表す `string` 型の変数を、ダブルクォート・シングルクォート・テンプレートリテラルの3通りで作ります。覚えてほしいのは `const 変数名: string = 値` という型注釈の形—— `: string` が「この箱には文字列しか入れません」という宣言です。

```
// =====
// string 型のサンプル - 「文字列」を意味する型
// =====
// const は「再代入禁止の変数宣言」。一度値を入れたら別の値で上書きできない。
// 変数名の右の : string が「型注釈」。「この変数は string 型ですよ」という宣言。
// = の右側が「初期値」。

// (1) ダブルクォート " で囲んだ文字列
//      greeting という名前の string 型変数に "こんにちは" を入れる。
const greeting: string = "こんにちは";

// (2) シングルクォート ' でも書ける (JavaScript/TypeScriptでは両者同じ意味)
//      チームのコーディング規約で「" に統一」「' に統一」を決めるのが普通。
const name: string = '太郎';

// (3) テンプレートリテラル: バッククォート ` で囲み、${変数名} で値を埋め込める
//      greeting と name の中身が連結されて入る。
//      文字列連結を + で書くより読みやすい ("こんにちは" + ", " + name + "さん!" よりずっと短い)
const message: string = `${greeting}、${name}さん!`;

// (4) console.log は「ターミナルやブラウザのコンソールに値を表示する」関数
//      第1引数で渡された文字列を1行表示する。
console.log(message);
// ▼ 実行結果 (ターミナルにこう出る)
// こんにちは、太郎さん！

// (5) テンプレートリテラルは改行もそのまま含められる
//      これが文字列の中に改行コード(\n)として保持される。
const multiLine: string = `
  1行目
  2行目
  3行目
`;
console.log(multiLine);
// ▼ 実行結果
// (空行)
//   1行目
//   2行目
//   3行目
// (空行)
```

```
// =====
// string 型でエラーになる例 - VS Code が赤線で警告してくれる
// =====

// (1) 数値を string 型変数に入れようとしている → ✖
//     42 は数値リテラル。string が期待される場所には置けない。
const title: string = 42;
// ▼ エラーメッセージ
// Type 'number' is not assignable to type 'string'.
//     (number 型は string 型に代入できません)

// (2) 真偽値を string 型変数に入れようとしている → ✖
const flag: string = true;
// Type 'boolean' is not assignable to type 'string'.

// (3) null を string 型変数に入れようとしている → ✖
//     ※ tsconfig.json で "strictNullChecks": true のとき
//     null や undefined はそれぞれ独立した型として扱われる。
const nothing: string = null;
// Type 'null' is not assignable to type 'string'.
```

2.2 number（数値型）

整数・浮動小数点数の両方を含む数値型です。JavaScript と同様に、TypeScript の number はすべて浮動小数点数として扱われます。

▼ このコードがやること（先に日本語で）：「数値」を表す **number** 型に入れられる色々な値——整数・小数・負の数・16進数・特殊な値（**Infinity** や **NaN**）など——を一通り作って表示します。今は「整数も小数も同じ **number** 型」という点だけ押さえれば十分で、特殊な記法はコメントを眺めるだけでOKです。

```
// =====
// number 型のサンプル - 「数値（整数も小数も）」を意味する型
// =====

// (1) 整数
const age: number = 25;

// (2) 小数（浮動小数点数）
const price: number = 1980.5;

// (3) 負の数
const negative: number = -10;

// (4) 16進数（0x で始まる）。Web開発で色コードを書くときに使う
//      0xff の f は 15 を意味する1桁の16進文字。ff = 15*16 + 15 = 255
const hex: number = 0xff;           // 256 - 1 = 255

// (5) 2進数（0b で始まる）。フラグ操作などで使う
//      0b1010 = 1*8 + 0*4 + 1*2 + 0*1 = 10
const binary: number = 0b1010;     // = 10

// (6) 8進数（0o で始まる）。Linuxのファイル権限で見る形
//      0o744 = 7*64 + 4*8 + 4*1 = 484
const octal: number = 0o744;       // = 484

// (7) 数値リテラル区切り（_ で読みやすくする）
//      1_000_000 と書いても 1000000 と書いても結果は同じ。実行時の値に _ は含まれない。
const big: number = 1_000_000;     // = 1000000（百万）

// (8) 特殊な数値
//      Infinity = 無限大、NaN = Not a Number（不正な数値計算の結果）
const inf: number = Infinity;
const notANumber: number = NaN;

console.log(age, price, negative, hex, binary, octal, big);
// ▼ 実行結果
// 25 1980.5 -10 255 10 484 1000000

console.log(1 / 0);           // → Infinity（0で割ると無限大）
console.log(0 / 0);           // → NaN（0÷0 は数学的に未定義）
console.log("a" * 2);         // → NaN（文字列を数値として掛けると NaN）
```

```
// =====
// number 型でエラーになる例
// =====

// (1) 数字に見えるが文字列なのでダメ → ✖
const count: number = "5";
// Type 'string' is not assignable to type 'number'.

// (2) "100" + 50 は JavaScript 的には string になる ("10050"になる) → TS は型エラー扱い
const total: number = "100" + 50;
// Type 'string' is not assignable to type 'number'.
//
// ※ TypeScript はこれを書く前にエラーで止めてくれる。
//   JavaScript だけだと、後の計算で予想外のバグになる。

// (3) 数値型変数 a と文字列型変数 b を + するとどうなる?
//     → JavaScript では "10" + "20" のように文字列連結に化ける。
//     TypeScript は「これは number として使えませんよ」と止めてくれる。
const a: number = 10;
const b: string = "20";
const result: number = a + b;
// Type 'string' is not assignable to type 'number'.
```

2.3 boolean (真偽値型)

`true` または `false` のどちらかを持つ型です。

▼ このコードがやること (先に日本語で): 「はい/いいえ」を表す `boolean` 型 (`true` か `false` の2択) を扱います。直接 `true` / `false` を入れるだけでなく、`age >= 18` のような比較式の結果もそのまま `boolean` になる、という点に注目してください。条件判定の土台になる型です。

```
// =====
// boolean 型のサンプル - 「true か false」しか入らない型
// =====

// (1) 直接 true / false を入れる
const isActive: boolean = true;      // true = 「アクティブ」
const isCompleted: boolean = false;  // false = 「未完了」

// (2) 比較式の結果も boolean になる
//     >= は「以上」、=== は「等しい (型もチェック)」
//     式が成り立てば true、成り立たなければ false が返る。
const isAdult: boolean = age >= 18;   // age が 25 なので true
const isEmpty: boolean = name === ""; // name が "太郎" なので false

// (3) 論理演算子 (&& と ||)
//     && (AND) : 両方 true なら true、片方でも false なら false
//     || (OR)  : 片方でも true なら true、両方 false なら false
//     ! (NOT)  : true ⇔ false を反転
const canAccess: boolean = isActive && isAdult; // true && true = true

console.log(isActive, isCompleted, isAdult, isEmpty, canAccess);
// ▼ 実行結果
// true false true false true
```

```
// =====
// boolean 型でエラーになる例
// =====

// (1) 数字 1 は「真っぽい値」だが boolean 型ではない → ✖
const flag: boolean = 1;
// Type 'number' is not assignable to type 'boolean'.

// (2) "true" は文字列であって、真偽値ではない → ✖
const active: boolean = "true";
// Type 'string' is not assignable to type 'boolean'.

// (3) 0 も「偽っぽい値」だが boolean 型ではない → ✖
//     JavaScript では if (0) {...} は実行されない (falsy扱い) が、
//     boolean 型としては number の 0 と boolean の false は別物。
const truthy: boolean = 0;
// Type 'number' is not assignable to type 'boolean'.
```

2.4 array (配列型)

同じ型の要素を複数格納するコレクションです。書き方は2通りあります。

▼ このコードがやること（先に日本語で）：「同じ型の値を順番に並べたリスト」である配列型を作り、追加（`push`）・取り出し（`[添え字]`）・長さ（`.length`）といった基本操作を試します。書き方は `string[]` と `Array<string>` の2通りありますが、どちらも意味は同じ。よく使うのは前者の `型名[]` です。

```
// =====
// 配列型のサンプル - 「同じ型の値を順番に並べたリスト」
// =====

// ----- 書き方1: 型名 + [] -----
// string[] は「string が並んだ配列」の意味。一番一般的な書き方。

// (1) 文字列の配列
const fruits: string[] = ["りんご", "みかん", "バナナ"];
//                               [0]       [1]       [2]   ← 添え字 (0から始まる)

// (2) 数値の配列
const scores: number[] = [85, 90, 78, 92];

// (3) 真偽値の配列
const flags: boolean[] = [true, false, true];

// ----- 書き方2: Array<型名> -----
// 結果は [] 記法と同じ。読み手の好みで選ぶ。

const colors: Array<string> = ["赤", "青", "緑"];
const ages: Array<number> = [20, 25, 30];

// ----- 配列の主な操作 -----

// (4) push: 配列の末尾に要素を追加 (破壊的に変更される)
//       型が一致する値だけ受け付ける (string配列に number は入らない)
fruits.push("ぶどう");           // fruits = ["りんご", "みかん", "バナナ", "ぶどう"]
scores.push(100);                // scores = [85, 90, 78, 92, 100]

// (5) [添え字] でアクセス。存在しない添え字は undefined (厳密モードでは型エラー)
const first: string = fruits[0]; // = "りんご"
console.log(first);
// ▼ 実行結果
// りんご

// (6) 配列の長さを取る
console.log(fruits.length);
// ▼ 実行結果
// 4

// (7) 空配列を作るときも型注釈を書いておくのが安全
//       こうしないと never[] と推論されてしまい、後で push できない。
const emptyStrings: string[] = [];
emptyStrings.push("hello");      // OK
console.log(emptyStrings);
```



```
// ▼ 実行結果
// [ 'hello' ]
```

▼ コードを1つずつ分解して解説

上のサンプルには、配列を扱ううえで初心者がつまづきやすいポイントがいくつかあります。順番に、1つずついねいに見ていきましょう。

解説1: 配列の型の書き方（**型名[]**）

```
const fruits: string[] = ["りんご", "みかん", "バナナ"];
//                        [0]      [1]      [2]      ← 添え字 (0から始まる)
```

- `const fruits` ... `const`（再代入禁止の変数）で `fruits` という名前の箱を作っています。
- `: string[]` ... これが配列の型注釈です。`string[]` は「`string`（文字列）がいくつも並んだ配列」という意味で、**型名** のうしろに `[]`（角カッコ）を付けると「その型の配列」を表します。
- `= ["りんご", ...]` ... `[]` の中に値を `,`（カンマ）で並べると**配列リテラル**（配列そのものの書き方）になります。
- 各要素には先頭から `0, 1, 2, ...` と添え字（インデックス）が振られます。`fruits[0]` が `"りんご"`、`fruits[1]` が `"みかん"` です。1番目が `0` から始まる点が最初は戸惑いやすいので要注意です。

用語: 添え字（インデックス） ... 配列の中の「何番目か」を表す番号。プログラミングでは **0 から数える**のが基本です（1ではありません）。

解説2: もう1つの書き方（**Array<型名>**）

```
const colors: Array<string> = ["赤", "青", "緑"];
const ages: Array<number> = [20, 25, 30];
```

- `Array<string>` は、解説1の `string[]` とまったく同じ意味です。「文字列の配列」を別の書き方で表しているだけです。
- `< >`（山カッコ）の中に要素の型を書きます。これは後の「ジェネリクス」（5章）で出てくる書き方の一種です。
- どちらを使ってもOKですが、よく使われるのは短い `string[]` のほうです。「`Array<...>` という書き方もあるんだな」と知っておけば十分です。

解説3: 配列に要素を追加する (`push`)

```
fruits.push("ぶどう");           // fruits = ["りんご", "みかん", "バナナ", "ぶどう"]
scores.push(100);                 // scores = [85, 90, 78, 92, 100]
```

- `.push(値)` は「配列の末尾（いちばん後ろ）に値を1つ追加する」メソッドです。
- 大事なのは、`push` は元の配列そのものを書き換える点です（これを「破壊的」と言います）。新しい配列を作るのではなく、`fruits` 自身が変わります。
- 型注釈のおかげで、`string[]` の配列には文字列しか `push` できません。`fruits.push(42)` のように数値を入れようとすると、TypeScript が即エラーで教えてくれます。

用語: メソッド ... 値（ここでは配列）が持っている「機能（関数）」のこと。配列 `.push(...)` のように `.`（ドット）でつないで呼び出します。

解説4: 要素の取り出しと長さ

```
const first: string = fruits[0]; // = "りんご"
console.log(first);
// ▼ 実行結果
// りんご

console.log(fruits.length);
// ▼ 実行結果
// 4
```

- `fruits[0]` ... `[ ]` に添え字を書くと、その位置の要素を取り出せます。`[0]` は1番目（`"りんご"`）です。
- `fruits.length` ... `.length`（レングス）は「その配列に要素が何個入っているか」を表すプロパティです。解説3で `"ぶどう"` を追加したので、要素数は `4` になっています。
- `console.log(...)` は、カッコの中の値をターミナルやブラウザのコンソールに表示する関数です。

用語: プロパティ ... 値が持っている「データ（情報）」のこと。`.length` のように `.` でつないで読み取ります。`()` を付けないのがメソッドとの違いです。

解説5: 空の配列には必ず型注釈を付ける

```
const emptyStrings: string[] = [];  
emptyStrings.push("hello");       // OK
```

- 最初は空っぽ（`[]`）の配列を作りたいことがあります。

- このとき型注釈 `: string[]` を必ず書くのがコツです。書かないと、TypeScript は中身が空で型を判断できず `never[]`（何も入れられない配列）と推論してしまい、あとで `push` できなくなります。
- 型注釈を書いておけば、「これは文字列を入れていく配列だ」と TypeScript に伝わり、`emptyStrings.push("hello")` が正しく通ります。

```
// --- エラーになる例 ---
const numbers: number[] = [1, 2, "three"]; // number[] と宣言しているのに
"three" は string。混在は許されない。
// エラー: Type 'string' is not assignable to type 'number'

const items: string[] = ["a", "b", "c"]; // string 専用配列。
items.push(42); // 42 は number なので入れられない。
// エラー: Argument of type 'number' is not assignable to parameter of type 'string'

// 異なる型が混在する配列を number[] として定義
const mixed: number[] = [1, true, "hello"]; // true (boolean) も
"hello" (string) も number ではない → 2件エラーになる。
// エラー: Type 'boolean' is not assignable to type 'number'
// エラー: Type 'string' is not assignable to type 'number'
```

2.5 tuple（タプル型）

タプル（tuple：複数の値を順序付きで束ねた、長さが固定のデータ構造）型 は、固定長で、各位置の型が決まっている配列 です。通常の配列と異なり、要素ごとに異なる型を持てます。

▼ このコードがやること（先に日本語で）：「**名前**、**年齢**」のように、**位置ごとに型と個数が決まった配列**である**タプル型**を作って使います。普通の配列と違い「1番目は文字列・2番目は数値」と並びが固定される点がポイントです。座標 **緯度**、**経度** のように、決まった組み合わせを表すのに便利です。

```
// --- 正しい例 ---
```

```
// [名前, 年齢] のタプル
```

```
const person: [string, number] = ["田中", 30]; // 型注釈の [string, number]
が「タプル型」。1番目=string, 2番目=number と位置で型が固定される。
```

```
// [ID, 名前, アクティブフラグ] のタプル
```

```
const record: [number, string, boolean] = [1, "太郎", true]; // 3要素のタプル。位置ごとに型
が決まっている。
```

```
// 要素へのアクセス（型が正しく推論される）
```

```
const personName: string = person[0]; // "田中" ← string 型 // [0] は1番目の要素を取り出す
（配列・タプルともに添え字は 0 始まり）。TypeScript は「1番目は string」と知っているので型が
string になる。
```

```
const personAge: number = person[1]; // 30 ← number 型 // [1] は2番目。number と推論さ
れる。
```

```
// 分割代入も可能
```

```
const [name, age] = person; // 配列形式の分割代入。
```

```
person[0] → name, person[1] → age に代入。型もそれぞれ string, number に決まる。
```

```
// name は string 型、age は number 型
```

```
// オプションなタプル要素
```

```
const optionalTuple: [string, number?] = ["田中"]; // 2要素目に ? を付けると「省
略してもよい」タプル要素になる。number? は number | undefined と同じ意味。
```

```
// 2番目の要素はあってもなくてもよい
```

```
// --- エラーになる例 ---
```

```
const pair: [string, number] = [42, "田中"]; // 1番目が string でなければ
ならないのに number。2番目が number でなければならないのに string。位置の型と合わない。
```

```
// エラー: Type 'number' is not assignable to type 'string' (1番目)
```

```
// エラー: Type 'string' is not assignable to type 'number' (2番目)
```

```
const triple: [string, number, boolean] = ["太郎", 25]; // タプル型は「長さも固定」。3
要素必要なのに 2要素しかないとエラー。
```

```
// エラー: Source has 2 element(s) but target requires 3
```

```
// 型が異なる位置へのアクセス
```

```
const data: [string, number] = ["hello", 42]; // 1番目が string、2番目が
number。
```

```
const wrongType: number = data[0]; // [0] は stringなのに
number に代入しようとしているのでエラー。
```

```
// エラー: Type 'string' is not assignable to type 'number'
```

2.6 object（オブジェクト型）

キーと値のペアを持つ構造化されたデータ型です。

▼ このコードがやること（先に日本語で）：「キー：値」の組をまとめた**オブジェクト型**を、`{ name: string; age: number }` のように形を指定して作ります。あわせて、省略可能な `?`（オプション）、変更禁止の `readonly`、オブジェクトの入れ子（ネスト）も扱います。「どんなプロパティを持つか」を型で表せるのがポイントです。

```
// --- 正しい例 ---

// オブジェクトリテラル型
const user: { name: string; age: number; email: string } = { // { ... } 中の「キー:
型」の並びが「オブジェクト型」の宣言。3つの必須プロパティを持つことを意味する。区切りは ; (または ,
でもOK)。
  name: "田中太郎", // string プロパティ。
  age: 30, // number プロパティ。
  email: "tanaka@example.com", // string プロパティ。
};

// オptionalプロパティ (? をつける)
const product: { name: string; price: number; description?: string } = { //
description? の ? は「あってもなくてもよい」を表す。
  name: "TypeScript入門書",
  price: 2980,
  // description は省略可能 // description は書かなく
ても OK。値は undefined になる。
};

// 読み取り専用プロパティ
const config: { readonly apiUrl: string; readonly timeout: number } = { // readonly キー
ワード = 「代入後は変更禁止」。config.apiUrl = ... と書くとエラーになる。
  apiUrl: "https://api.example.com",
  timeout: 5000,
};

// ネストしたオブジェクト
const company: { // 型注釈の中にも { ...
} を入れ子にできる。
  name: string;
  address: { // address 自体がオブジ
エクト型。
    city: string;
    zipCode: string;
  };
} = {
  name: "サンプル株式会社",
  address: {
    city: "東京",
    zipCode: "100-0001",
  },
};
```

▼ コードを1つずつ分解して解説

オブジェクト型には、初心者がつまづきやすい書き方（オプション・`readonly`・ネスト）が詰まっています。順番に、1つずついねいに見ていきましょう。

解説1: オブジェクト型の基本の書き方

```
const user: { name: string; age: number; email: string } = {  
  name: "田中太郎",  
  age: 30,  
  email: "tanaka@example.com",  
};
```

- `: { name: string; age: number; email: string }` の部分が**オブジェクトの型注釈**です。`{ }` の中に「**キー名: 型**」の組を並べて、「このオブジェクトはどんな項目を持つか」を宣言します。
- ここでは「`name`（文字列）・`age`（数値）・`email`（文字列）の3つを必ず持つ」という意味になります。区切りは `;`（セミicolon）でも `,`（カンマ）でもOKです。
- `= { name: "田中太郎", ... }` が実際の値（オブジェクトリテラル）。**型側の `{ }`** は「形の宣言」、**値側の `{ }`** は「中身そのもの」という違いに注意してください。
- 型で宣言した項目が1つでも欠けていたり、型が違ったりすると、TypeScript が即エラーで教えてくれます。

用語: プロパティ（キーと値） ... オブジェクトの中の「`name: "田中太郎"`」のような1組のこと。左の `name` を「キー（項目名）」、右の `"田中太郎"` を「値」と呼びます。

解説2: オプションalプロパティ（?）

```
const product: { name: string; price: number; description?: string } = {  
  name: "TypeScript入門書",  
  price: 2980,  
  // description は省略可能  
};
```

- キー名のうしろに付いている `?`（`description?`）が**オプションal**の印です。「この項目はあってもなくてもよい」という意味になります。
- そのため上の例では `description` を書いていませんが、エラーになりません。
- 省略した場合、`product.description` の値は `undefined`（値が無い状態）になります。
- 「必須にしたいなら `?` なし、任意にしたいなら `?` あり」と覚えておきましょう。

用語: オプションal ... 「省略可能」という意味。`?` を付けると、その項目は書いても書かなくてもよくなります。

解説3: 読み取り専用プロパティ (`readonly`)

```
const config: { readonly apiUrl: string; readonly timeout: number } = {
  apiUrl: "https://api.example.com",
  timeout: 5000,
};
```

- キー名の前に付く `readonly` (リードオンリー) は「一度値を入れたら、あとから変更できない」という印です。
- 例えば `config.apiUrl = "別のURL"` と書こうとすると、TypeScript がエラーで止めてくれます。
- 「設定値 (API の URL など) のように、途中で書き換わってほしくない項目」に付けると、うっかり変更によるバグを防げます。

用語: `readonly` ... 「読み取り専用」。値を読むことはできるが、あとから書き換える (再代入する) ことはできません。

解説4: ネスト (入れ子) したオブジェクト

```
const company: {
  name: string;
  address: {
    city: string;
    zipCode: string;
  };
} = {
  name: "サンプル株式会社",
  address: {
    city: "東京",
    zipCode: "100-0001",
  },
};
```

- オブジェクトの中に、さらにオブジェクトを入れることを **ネスト (入れ子)** と呼びます。
- ここでは `company` の中の `address` が、それ自体「`city` と `zipCode` を持つオブジェクト」になっています。
- 型注釈の側も、値の側も、`{ }` の中に `{ }` を書く「入れ子」の形になっている点に注目してください。形 (型) と中身 (値) が対応しています。
- 取り出すときは `company.address.city` (東京) のように、`.` (ドット) をつなげて深い階層にアクセスします。

用語: **ネスト (入れ子) ...** あるものの中に、同じ種類のものをさらに入れること。オブジェクトの中のオブジェクト、配列の中の配列、などを指します。


```
// --- エラーになる例 ---
const user: { name: string; age: number } = {
  name: "田中",
  // age がない!
};
// エラー: Property 'age' is missing in type '{ name: string; }'

const item: { name: string; price: number } = {
  name: "ペン",
  price: 100,
  color: "赤", // 余分なプロパティ
};
// エラー: Object literal may only specify known properties,
// and 'color' does not exist in type '{ name: string; price: number; }'

// readonly プロパティへの再代入
const settings: { readonly theme: string } = { theme: "dark" }; // readonly なので
// theme は読み取り専用。
settings.theme = "light";
// エラー: Cannot assign to 'theme' because it is a read-only property
```

2.7 any 型

すべての型チェックを無効化する 型です。どんな値でも受け入れ、どんな操作も許可します。

▼ このコードがやること（先に日本語で）：「何でもアリ」になってしまう **any** 型の挙動を体験します。**any** を使うとどんな値も代入でき、存在しないメソッド呼び出しすらエラーになりません——つまり TypeScript の安全装置が全部オフになります。**any** は原則使わない、と心に刻むための例です。

```
// --- any の使用例（非推奨だが動く） ---
let anything: any = "文字列"; // any は「何でもアリ型」。型チェックを全部スキップする魔法のキーワード（=危険）。

anything = 42; // OK (number を代入) // どんな型の値も再代入できる。
anything = true; // OK (boolean を代入) // ↑同じく。
anything = [1, 2, 3]; // OK (配列を代入) // ↑同じく。
anything.foo(); // OK (存在しないメソッドも呼べる → 実行時エラー!) // foo というメソッドは無いが、TypeScript は何も言わない。実際に動かすと TypeError で落ちる。
anything.bar.baz; // OK (存在しないプロパティもアクセス可能 → 実行時エラー!) // .bar が undefined、その .baz を読もうとして実行時にクラッシュ。
```

警告: `any` を使うと TypeScript を使う意味がほぼなくなります。原則として `any` は使わないでください。やむを得ない場合（外部ライブラリの型定義がない場合など）のみ、限定的に使います。

```
// --- なぜ any が危険か ---
function processData(data: any) { // 引数 data の型を any
    にしてしまうと...
    // 型チェックが一切行われない
    return data.toUpperCase(); // data が string でなければ実行時エラー // .toUpperCase() は
    string のメソッド。data が string でないと実行時に死ぬ。
}

processData("hello"); // OK: "HELLO" // 文字列なので OK。
processData(42); // 実行時エラー: data.toUpperCase is not a function // 数値に
toUpperCase は無い。
processData(null); // 実行時エラー: Cannot read properties of null // null の
.toUpperCase を読もうとして即死。
```

2.8 unknown 型

`any` の安全な代替です。どんな値も代入できますが、**使う前に型チェックが必要**です。

▼ このコードがやること（先に日本語で）：`any` の安全版である `unknown` 型を使います。どんな値も入れられる点は `any` と同じですが、**使う前に「これは文字列？」と型を確かめないと触れないのが違い**です。`typeof` で自身の型を確認してから使う、という流れ（型ガード）に注目してください。

```

// --- unknown の正しい使い方 ---
let value: unknown = "こんにちは"; // unknown 型変数を宣言。「中身は
何か分からない」を明示する型。
value = 42; // OK (代入は自由) // どんな型でも代入はできる (ここは
any と同じ)。
value = true; // OK

// ただし、そのまま使うことはできない
// const upper: string = value.toUpperCase(); // ↑ unknown のままだとメソッド呼
び出しが禁止される (安全性のため)。
// エラー: 'value' is of type 'unknown'

// 型チェック (型ガード) を行ってから使う
if (typeof value === "string") { // typeof は値の型を文字列で返す
演算子。"string"|"number"|"boolean"|"undefined"|"object"|"function"|"symbol"|"bigint" の
いずれか。
    // このブロック内では value は string 型として扱える // 条件式により TypeScript が「ここ
では string と確定」と推論する。これを「型の絞り込み (narrowing)」と呼ぶ。
    console.log(value.toUpperCase()); // OK // string なので toUpperCase
が呼べる。
}

if (typeof value === "number") {
    // このブロック内では value は number 型として扱える
    console.log(value.toFixed(2)); // OK // .toFixed(2) は number のメ
ソッドで「小数点以下2桁の文字列に変換」する。
}

```

```

// --- any と unknown の比較 ---

// any: 危険 (型チェックなし)
function unsafeProcess(data: any): string { // 引数を any にすると...
    return data.toUpperCase(); // 実行時に壊れる可能性あり // 何でも通るが安全性ゼロ。
}

// unknown: 安全 (型チェック必須)
function safeProcess(data: unknown): string { // unknown にすると...
    if (typeof data === "string") { // 必ず型ガードを書かないと中
身を使えない。
        return data.toUpperCase(); // 型チェック済みなので安全 // string と確定したブロックなの
で OK。
    }
    return "不明なデータ"; // 型が string でなければデフ
ォルト値を返す。
}

```

2.9 never 型

決して発生しない値 の型です。主に以下の場面で使われます。

▼ このコードがやること（先に日本語で）：「決して値を返さない」ことを表す `never` 型の使いどころを3つ見ます。
①必ず例外を投げる関数、②終わらない無限ループ、③ `switch` で全パターンを処理したか確認する「網羅性チェック」です。最初は③が少し難しいので、①②で「正常に終わらない関数の型なんだな」とつかめれば十分です。

```

// --- 用途1: 絶対に値を返さない関数 ---
function throwError(message: string): never {
    「決して return しない」関数の印。
    throw new Error(message);
    文。new Error(...) で Error オブジェクトを作る。throw すると関数はここで終わるので、return されない。
    // この関数は必ず例外を投げるので、正常に値を返すことがない
}

// --- 用途2: 無限ループ ---
function infiniteLoop(): never {
    never。
    while (true) {
        の間ずっと繰り返し = 無限ループ。
        // 永遠に終わらない
    }
}

// --- 用途3: 到達不可能なコードの検出 (網羅性チェック) ---
type Shape = "circle" | "square" | "triangle";
// ユニオン型で3種類のリテラルを宣言。

function getArea(shape: Shape): number {
    返す関数。
    switch (shape) {
        case "circle":
            とき。
            return Math.PI * 10 * 10;
            半径10の円の面積を返す。
        case "square":
            return 10 * 10;
        case "triangle":
            return (10 * 10) / 2;
        default:
            // すべてのケースを処理済みなら、shape は never 型になる // 上の case で全てのリテラルを
            処理し尽くしているので、ここに来る可能性はなく shape の型は never に絞り込まれる。
            const _exhaustiveCheck: never = shape;
            代入。これが「網羅性チェック」の魔法。
            return _exhaustiveCheck;
            代入可 (never はすべての型の部分型)。
    }
}

```

// 戻り値の型 : never は

// throw = 例外を投げる

// ずっと終わらない関数も

// while (true) は「true

// shape を受け取って面積を

// switch 文: 値ごとに分

// shape が "circle" の

// Math.PI は 円周率 π 。

// 1辺10 の正方形の面積。

// 底辺10・高さ10の三角形

// never 型に shape を

// never は number にも

```
// --- never 型の重要性: 網羅性チェック ---

// Shape に新しい種類を追加した場合
type Shape = "circle" | "square" | "triangle" | "pentagon"; // pentagon を追加 // ↑ あとから1種類追加した、と想像してください。

function getArea(shape: Shape): number {
  switch (shape) {
    case "circle":
      return Math.PI * 10 * 10;
    case "square":
      return 10 * 10;
    case "triangle":
      return (10 * 10) / 2;
    default:
      // "pentagon" のケースが未処理なのでエラーになる！ // default に来る可能性が
      // 「pentagon」として残っているの、shape は "pentagon" 型に絞り込まれる。
      const _exhaustiveCheck: never = shape; // "pentagon" は never
      // に入力できないのでエラー → 抜け漏れに気づける！
      // エラー: Type 'string' is not assignable to type 'never'
      // → "pentagon" の処理を追加し忘れたことに気づける
      return _exhaustiveCheck;
  }
}
```

2.10 void 型

値を返さない関数 の戻り値の型です。 `never` と違い、関数自体は正常に終了します。

▼ このコードがやること（先に日本語で）: 「何も値を返さない関数」の戻り値に付ける `void` 型を使います。
`console.log` するだけ・`alert` を出すだけ、といった「処理はするが結果を返さない」関数の印です。 `never` と違って関数自体はちゃんと正常に終わる、という点が違います。

```
// --- 正しい例 ---
function logMessage(message: string): void {
    console.log(message);
    // return 文がない、または return; のみ
    // している扱い。
}

function showAlert(text: string): void {
    alert(text);
    // alert はブラウザのポップアップ表示関数 (Node.js には無い)。
    return; // return; は OK (値を返さない)
    // ければ void と矛盾しない。
}

// void 型の変数には undefined のみ代入可能
const result: void = undefined;
// void 変数に入れられるのは undefined のみ。
```

// 戻り値 : void = 「値を返さない関数」。

// 文字列を表示するだけ。
// 暗黙的に undefined を返し

// alert はブラウザのポップアップ表示関数 (Node.js には無い)。
// return の後に値を書かなければ void と矛盾しない。

// void 変数に入れられるのは undefined のみ。

```
// --- エラーになる例 ---
function greet(name: string): void {
    return `こんにちは、${name}さん`;
}

// エラー: Type 'string' is not assignable to type 'void'
// void なのに値を返している

const value: void = "hello";
// エラー: Type 'string' is not assignable to type 'void'

const num: void = 42;
// エラー: Type 'number' is not assignable to type 'void'
```

// void と宣言しているのに...

// 文字列を return しているので矛盾。

// void に文字列を入れられない。

// void に数値も入れられない。

2.11 null と undefined

null (ヌル：明示的な「無」) は「値が意図的に空」であることを表し、**undefined** (アンディファインド：未定義) は「値が未定義」であることを表します。

使い分けの目安: - `undefined` : 自然発生する「値がない」。変数を宣言しただけで初期化していない、関数が何も return しないとき、オブジェクトの存在しないプロパティを読んだとき。 - `null` : プログラムが「ここは意図的に空ですよ」と明示するときに使う。例えば API で「ユーザーが見つからない」場合に `null` を返すなど。 - **迷ったら `undefined` を使う** のが TypeScript の流儀。`null` は外部から値が来る場合（API, DB など）に限定するのが一般的。

▼ **このコードがやること（先に日本語で）** : 「値が無い」を表す `null` / `undefined` を安全に扱う方法を学びます。`string | null`（文字列または空）のように型で「無いかも」と明示し、使う前に `if (user !== null)` で確認したり、`?.`（オプショナルチェイニング）や `??`（無いときの代替値）で安全に触ったりします。クラッシュを防ぐ大事なテクニックです。


```

// --- 正しい例 ---

// strictNullChecks が有効な場合 (推奨)
let nullableString: string | null = null; // 明示的に null を許可 // string | null
// 「string または null」の意。| はユニオン型の区切り。
let optionalValue: number | undefined = undefined; // number また
// は undefined を許可する変数。

// null チェック
function findUser(id: number): string | null { // 戻り値が
  // 「string または null」の関数。
  if (id === 1) { // === は厳密
    // 等価。id が 1 のとき。
    return "田中太郎";
  }
  return null; // ユーザーが見つからない場合 // 1 以外は明示的に
// null を返す。
}

const user = findUser(999); // user の型
// は string | null と推論される。
if (user !== null) { // null でな
  // いことを確認する型ガード。
  console.log(user.toUpperCase()); // null チェック後なので安全 // このブロック内
  // では user は string と絞り込まれる。
}

// オプショナルチェイニング (?.)
const length = user?.length; // user が null なら undefined を返す // ?. は「左が
// null/undefined ならその場で undefined を返し、そうでなければ .length を読む」演算子。
// .length のクラッシュ回避用。

// null 合体演算子 (??)
const displayName = user ?? "ゲスト"; // user が null/undefined なら "ゲスト" // ?? は「左
// が null か undefined なら右の値を使う」演算子。|| と似ているが、|| は 0 や "" も false 扱いする
// のに対し ?? は null/undefined だけを判定する。

```

```
// --- エラーになる例 (strictNullChecks 有効時) ---
let name: string = null; // string 型
に null は入らない (strictNullChecks: true の場合)。
// エラー: Type 'null' is not assignable to type 'string'

let age: number = undefined; // number
型に undefined も入らない。
// エラー: Type 'undefined' is not assignable to type 'number'

// null チェックなしでのメソッド呼び出し
function findItem(id: number): string | null {
    return id > 0 ? "アイテム" : null; // 三項演算子
};。 「条件 ? trueのとき : falseのとき」の形。
}

const item = findItem(-1); // item の型
は string | null。
console.log(item.toUpperCase()); // チェックなし
しで .toUpperCase() を呼ぶとエラー。
// エラー: 'item' is possibly 'null'
// → item が null の可能性があるので、チェックなしでは使えない
```

基本型のまとめ表

型	説明	例	使用場面
<code>string</code>	文字列	<code>"hello", 'world'</code>	テキストデータ全般
<code>number</code>	数値	<code>42, 3.14</code>	数値計算、ID
<code>boolean</code>	真偽値	<code>true, false</code>	フラグ 条件
<code>string[]</code>	文字列配列	<code>["a", "b"]</code>	リストデータ
<code>[string, number]</code>	タプル	<code>["太郎", 25]</code>	固定構造のペア
<code>object</code>	オブジェクト	<code>{ key: "value" }</code>	構造化データ
<code>any</code>	なんでも	すべて	使用非推奨
<code>unknown</code>	不明な型	すべて	安全な any の代替
<code>never</code>	発生しない	なし	網羅性チェック
<code>void</code>	返り値なし	<code>undefined</code>	関数の戻り値
<code>null</code>	空	<code>null</code>	意図的な空値
<code>undefined</code>	未定義	<code>undefined</code>	未初期化の値

3. 型注釈と型推論

3.1 明示的な型注釈

型注釈（Type Annotation：タイプアノテーション。変数や関数の引数・戻り値に「これは何の型か」を明示的に書くこと）とは、変数や関数のパラメータ・戻り値に明示的に型を記述することです。コロンの（:）の後に型を書きます。

▼ このコードがやること（先に日本語で）：変数・関数の引数・戻り値などに「これは何の型か」を自分で明示的に書く（型注釈）やり方を一通り見ます。共通ルールは「**名前のうしろに : を付けて型を書く**」こと——

`bookTitle: string`、`(price: number): number` のようにです。形に慣れるのが目的です。

```
// 変数の型注釈
const bookTitle: string = "TypeScript入門";           // 変数名 : 型 = 値
// 順。: string が型注釈。

const pageCount: number = 350;                         // : number で
// 「数値型」を宣言。

const isPublished: boolean = true;                     // : boolean で
// 「真偽値型」を宣言。

// 関数のパラメータと戻り値の型注釈
function calculateTotal(price: number, quantity: number): number { // (引数1: 型, 引
// 数2: 型): 戻り値の型 の形。
    return price * quantity;                               // price と
// quantity は両方 number なので、* の結果も number。
}

// アロー関数の型注釈
const add = (a: number, b: number): number => a + b;     // アロー関数版。(
// ): 戻り値の型 => 式 の形。

// オブジェクトの型注釈
const book: { title: string; author: string; pages: number } = { // { キー: 型;
// ... } の形でオブジェクト型を直接書ける。
    title: "TypeScript入門",
    author: "山田太郎",
    pages: 350,
};

// 配列の型注釈
const tags: string[] = ["プログラミング", "TypeScript", "入門"]; // string[] =
// 「string の配列」。
```

3.2 型推論の仕組み

TypeScript は非常に賢い **型推論**（Type Inference：タイプインファレンス。初期値や使われ方から TypeScript が自動的に型を判断する機能）エンジンを持っています。多くの場合、型注釈を書かなくても TypeScript が自動的に型を判断してくれます。

▼ このコードがやること（先に日本語で）：型注釈を書かなくても、TypeScript が初期値から**自動で型を判断してくれる**（型推論）様子を見ます。`const message = "..."` と書けば勝手に `string` と分かる、という具合です。「明らかな型はわざわざ書かなくてよい」のがポイント。詳しい推論結果はコメントに書いてあります。

```

// --- TypeScript が自動で型を推論する例 ---

// 変数の初期化から推論
const message = "こんにちは"; // string と推論 // 右辺が文字列リテラルなので、型注釈なしでも string と分かる。
const count = 42; // number と推論 // 数値リテラル → number。
const isValid = true; // boolean と推論 // true/false → boolean。
const items = [1, 2, 3]; // number[] と推論 // 配列の中身が全部 number なら number[] と推論。

// 関数の戻り値も推論される
function multiply(a: number, b: number) { // 戻り値の型注釈を省略しても...
    return a * b; // 戻り値は number と推論される // a * b が number なので戻り値は number と分かる。
}

// オブジェクトの構造も推論される
const user = {
    name: "田中", // name: string // 右辺の値から各プロパティの型が決まる。
    age: 25, // age: number
    isActive: true, // isActive: boolean
};
// user の型は { name: string; age: number; isActive: boolean } と推論

// 配列のメソッドから推論
const doubled = [1, 2, 3].map((n) => n * 2); // map の引数 n は配列の要素型から推論されて number。返す n * 2 も number。結果は number[]。
// doubled は number[] と推論

// 条件式から推論
const status = count > 10 ? "many" : "few"; // 三項演算子 ?.: 両辺が string なので結果は string と推論される。
// status は string と推論

```

```
// --- 型推論の注意点 ---
```

```
// let で宣言すると広い型に推論される
```

```
let color = "red"; // string と推論 ("red" リテラル型ではない) // let は後から再代  
入されるので「広めの型」(=string) に推論される。
```

```
color = "blue"; // OK (string なので他の文字列も代入可能)
```

```
// const で宣言するとリテラル型に推論される
```

```
const direction = "north"; // "north" と推論 (リテラル型) // const は変わら  
ないので「ピッタリの型」(="north") に推論される。
```

```
// direction = "south"; // エラー: const なので再代入不可
```

```
// 空配列は any[] に推論される (要注意)
```

```
const emptyList = []; // any[] と推論 // 中身がないの  
で TypeScript は要素型を決められず any[] になる (ゆるい)。
```

```
// → 型注釈をつけるべき
```

```
const emptyStrings: string[] = []; // 明示的に型を指定 // 必ず型注釈を書  
くのが安全。
```

3.3 いつ型注釈を書くべきか



型注釈を書くべき場面

▼このコードがやること（先に日本語で）：「型注釈を自分で書いたほうがよい5つの場面」を実例で示します。特に大事なのは①関数の引数——ここは TypeScript が推論できないので必ず型を書く、というルールです。空配列の初期化や複数の型を受け取る変数なども、書いておくと安全になります。

```

// 1. 関数のパラメータ (必須! 推論できない)
function greet(name: string, age: number): string { // 引数は値が無い段階
  // 書くので、TypeScript は推論できない。書かないと暗黙の any になる。
  return `${name}さんは${age}歳です`; // テンプレートリテラ
  // ルで挨拶文を生成。
}

// 2. 空配列を初期化する場合
const users: string[] = []; // 型注釈がないと any[] になる // 後で push する型を
// 限定したいなら必須。

// 3. 関数の戻り値を明確にしたい場合 (公開APIなど)
function fetchUser(id: number): Promise<User> { // Promise<T> は
  // 「将来 T を返す」を表す型。fetch は Promise を返すので戻り値も Promise になる。
  // 戻り値の型が明確になり、実装ミスを防げる
  return fetch(`/api/users/${id}`).then((res) => res.json()); // fetch はサーバー
  // に HTTP リクエストを送る関数。 .then で結果を加工する。
}

// 4. 複数の型を受け入れる場合
let result: string | number; // | はユニオン型の
// 区切り。「string か number のどちらか」。
result = "成功"; // string なので
// OK。
result = 404; // number なので
// OK。

// 5. オブジェクトの構造を明確にしたい場合
interface Config { // interface でオ
  // ブジェクトの型を定義。
  apiUrl: string;
  timeout: number;
  retries: number;
}

const config: Config = { // Config 型とし
  // て宣言。3つのプロパティが必須。
  apiUrl: "https://api.example.com",
  timeout: 5000,
  retries: 3,
};

```

型推論に任せてよい場面

▼このコードがやること（先に日本語で）：逆に「型注釈を書かなくてよい（推論に任せてよい）場面」を示します。初期値を見れば型が一目瞭然なときは、わざわざ書くとかえって冗長になります。「明らかなら省く、曖昧なら書く」というバランス感覚をつかむのが目的です。

```
// 1. 初期値から型が明らかな場合
const name = "田中太郎"; // 明らかに string // : string と書かなくても推論される。冗長さを避けるために省くのも良い設計。
const age = 30; // 明らかに number
const isActive = true; // 明らかに boolean

// 2. 配列リテラルで初期化する場合
const fruits = ["りんご", "みかん"]; // 明らかに string[] // 中身があれば推論できる。

// 3. 関数の戻り値が単純な場合
function add(a: number, b: number) { // 戻り値の型を省略。
    return a + b; // 明らかに number を返す // number + number
} // number と推論。

// 4. 変数の型が変わらない場合
const total = price * quantity; // 明らかに number // 計算結果から推論。
```

4. インターフェースと型エイリアス

4.1 interface の定義方法

interface はオブジェクトの「形（シェイプ）」を定義する方法です。

▼このコードがやること（先に日本語で）：オブジェクトの「形（どんなプロパティを持つか）」を定義する **interface**（設計図）の基本を学びます。**User** という型を作っておけば、その形に合わないオブジェクトは即エラーになります。あわせて、省略可能な **?**、変更禁止の **readonly** も扱います。書籍管理アプリでもこの形で **Book** 型を作っていきます。


```
// =====
// interface の基本 - 「オブジェクトの設計図」を作る
// =====
// interface は「このオブジェクトは こういう形をしてないとダメだよ」という
// ルールを宣言する仕組み。クラスの設計図のようなものを軽量に書ける。
// 大文字始りにするのが慣習（クラス名と同じ）。

// (1) User という名前の interface を定義
// 「User 型のオブジェクトには id, name, email, age の4つが必須」と宣言
interface User {
  id: number;      // 数値型のID（必須）
  name: string;    // 文字列型の名前（必須）
  email: string;   // 文字列型のメールアドレス（必須）
  age: number;     // 数値型の年齢（必須）
}

// (2) User 型の変数を作る
// プロパティが1つでも欠けるとエラー、余計なプロパティもエラー（厳格チェック）
const user: User = {
  id: 1,
  name: "田中太郎",
  email: "tanaka@example.com",
  age: 30,
};

console.log(user);
// ▼ 実行結果
// { id: 1, name: '田中太郎', email: 'tanaka@example.com', age: 30 }

// =====
// オptionalプロパティ「?」を使うと、あってもなくても良くなる
// =====

interface Product {
  id: number;      // 必須
  name: string;    // 必須
  price: number;   // 必須
  description?: string; // ← 「?」 付きなので省略可能。値が無いときは undefined
}

// description を省略しても OK
const pen: Product = {
  id: 1,
  name: "ボールペン",
  price: 150,
  // description は書いていないが、? なのでエラーにならない
};
```

```

console.log(pen.description);
// ▼ 実行結果
// undefined

// =====
// readonly: 「あとから変えられないプロパティ」
// =====

interface Config {
  readonly apiUrl: string;    // readonly = 後で代入禁止
  readonly timeout: number;
}

const config: Config = {
  apiUrl: "https://api.example.com",
  timeout: 5000,
};

// config.apiUrl = "https://other.com";    // ← これを書くとエラー
// ▼ エラー
// Cannot assign to 'apiUrl' because it is a read-only property.
//   ('apiUrl' は読み取り専用プロパティのため代入できません)

```

▼ コードを1つずつ分解して解説

interface を初めて見ると「クラス？ 型？」と混乱しがちです。順番に、1つずついねいに見ていきましょう。

解説1: **interface** で「オブジェクトの設計図」を作る

```

interface User {
  id: number;      // 数値型のID (必須)
  name: string;    // 文字列型の名前 (必須)
  email: string;   // 文字列型のメールアドレス (必須)
  age: number;     // 数値型の年齢 (必須)
}

```

- **interface User { ... }** は「**User** という名前の型（オブジェクトの設計図）を作る」という宣言です。
interface（インターフェース）は「このオブジェクトはこういう形をしていてね」というルールを定義する道具です。
- **{ }** の中に「キー名: 型」を1行ずつ並べます。ここでは **id**・**name**・**email**・**age** の4項目を持つと宣言しています。
- 各行のうしろには **;**（セミコロン）を付けますが、改行で区切るだけでも動きます。
- 型名は **大文字始まり**（**User**）にするのが慣習です。

用語: **interface** (インターフェース) ... オブジェクトの「形 (どんな項目を持つか)」を定義するための、TypeScript の道具のひとつ。 **type** と並んでよく使われます。

解説2: 定義した型を使ってオブジェクトを作る

```
const user: User = {
  id: 1,
  name: "田中太郎",
  email: "tanaka@example.com",
  age: 30,
};
```

- `: User` という型注釈で、「この `user` は `User` 型の形をしていますよ」と宣言しています。
- これにより、`User` で決めた4項目をすべて正しい型で持っていないとエラーになります。
- 1つでも項目が欠けるとエラー (例: `age` を書き忘れる)。
- 型が違ってエラー (例: `age: "30"` と文字列にする)。
- 余計な項目を足してもエラー (定義にないキーは許されません)。
- このように、一度 **interface** を作っておけば、同じ形のオブジェクトを安全に何個でも作れるのが利点です。

解説3: オプショナルプロパティ (?)

```
interface Product {
  id: number;           // 必須
  name: string;         // 必須
  price: number;        // 必須
  description?: string; // ← 「?」 付きなので省略可能。値が無いときは undefined
}
```

- キー名のうしろの **?** (`description?`) は「この項目は省略してもよい」という印 (オプショナル) です。
- そのため `description` を書かずにオブジェクトを作ってもエラーになりません。省略した場合の値は `undefined` (値が無い状態) になります。
- 「必ず持っていてほしい項目には **?** なし、あってもなくてもよい項目には **?** あり」と使い分けます。

解説4: 読み取り専用プロパティ (`readonly`)

```
interface Config {  
  readonly apiUrl: string;    // readonly = 後で代入禁止  
  readonly timeout: number;  
}
```

- キー名の前に付く `readonly` は「一度値を入れたら、あとから変更できない」という印です。
- 例えば `config.apiUrl = "https://other.com"` と書き換えようとすると、TypeScript が「読み取り専用なので代入できません」とエラーで止めてくれます。
- API の URL や設定値のように「途中で書き換わってほしくない項目」に付けると、うっかり変更によるバグを防げます。

▼ このコードがやること（先に日本語で）：既存の `interface` に項目を追加した新しい型を作る「継承（`extends`）」を学びます。`Dog extends Animal` と書けば「Animal の項目を全部受け継いだうえで、犬固有の項目を足す」という意味になり、同じ定義を何度も書かずに済みます。複数の型をまとめて継承することもできます。

```
// =====
// interface の継承 (extends) - 既存の型に項目を追加した新しい型を作る
// =====

// (1) ベースとなる Animal 型
interface Animal {
  name: string;    // 動物の名前
  age: number;     // 年齢
}

// (2) Dog は Animal を「extends」している
//     → Animal の name, age を全て持ったうえで、breed と isVaccinated が追加される。
//     継承元の項目は書かなくてもOK (自動で引き継がれる)。
interface Dog extends Animal {
  breed: string;    // 犬種 (Dog で追加)
  isVaccinated: boolean; // ワクチン接種済みか (Dog で追加)
}

// (3) Dog 型のオブジェクトには合計4つのプロパティが必須
const myDog: Dog = {
  name: "ポチ",      // Animal 由来
  age: 3,            // Animal 由来
  breed: "柴犬",     // Dog 固有
  isVaccinated: true, // Dog 固有
};
console.log(myDog.name, myDog.breed);
// ▼ 実行結果
// ポチ 柴犬

// =====
// 複数の interface を同時に継承 (カンマ区切り)
// =====
// 「ID を持つ」「タイムスタンプを持つ」のような共通の特徴を細かく分けて、
// 必要な組み合わせを extends で混ぜるのが現代的な設計。

// (1) 「ID を持つ」だけを表現する interface
interface HasId {
  id: number;
}

// (2) 「作成日時・更新日時を持つ」だけを表現する interface
interface HasTimestamp {
  createdAt: Date;    // Date は JavaScript 標準の日時オブジェクト
  updatedAt: Date;
}

// (3) BlogPost は HasId と HasTimestamp の両方を継承し、独自フィールドを追加
interface BlogPost extends HasId, HasTimestamp {
```

```

title: string;    // タイトル
content: string;  // 本文
author: string;   // 著者
}

// (4) BlogPost 型のオブジェクトには6つのプロパティが必須
//     new Date("2025-01-01") は「2025年1月1日 0:00 UTC」を表す Date オブジェクト
const post: BlogPost = {
  id: 1,
  createdAt: new Date("2025-01-01"),
  updatedAt: new Date("2025-06-15"),
  title: "TypeScript入門",
  content: "TypeScriptの基礎を学びましょう",
  author: "田中太郎",
};

console.log(post.title, "投稿日:", post.createdAt.toISOString());
// ▼ 実行結果
// TypeScript入門 投稿日: 2025-01-01T00:00:00.000Z

```

▼このコードがやること（先に日本語で）：**interface** には、データだけでなく「メソッド（関数）を持つ」という形も定義できます。ここでは四則演算の関数4つを持つ **Calculator** 型を作り、それに合わせて実装します。「この型のオブジェクトは、こういう関数を必ず持っていてね」と約束させられる、という点がポイントです。

```
// interface でメソッドを定義
interface Calculator {
  // 「メソッド（関数プロパティ）」の型定義。
  add(a: number, b: number): number;
  // add メソッドの型。引数2つ、戻り値 number。
  subtract(a: number, b: number): number;
  // 同様に subtract（引き算）。
  multiply(a: number, b: number): number;
  // multiply（掛け算）。
  divide(a: number, b: number): number;
  // divide（割り算）。
}

const calc: Calculator = {
  // Calculator 型として4つのメソッドを実装。
  add: (a, b) => a + b,
  // 各メソッドはアロー関数で実装。引数の型は interface から推論される。
  subtract: (a, b) => a - b,
  multiply: (a, b) => a * b,
  divide: (a, b) => {
    // 割り算は 0 除算チェックがあるので { } で複数行に。
    if (b === 0) throw new Error("0で割ることはできません");
    // b が 0 のとき例外を投げる。throw は実行を中断する文。
    return a / b;
    // 正常時は割った結果を返す。
  },
};
```

4.2 type の定義方法

type（型エイリアス）はあらゆる型に名前をつける方法です。

▼このコードがやること（先に日本語で）：**type**（型エイリアス）を使って、あらゆる型に自分で名前を付ける方法を学びます。**interface** がオブジェクトの形専用なのに対し、**type** は文字列の別名・ユニオン型（「AまたはB」）・タプル・関数型など、何にでも名前を付けられるのが強みです。ここでは代表的なパターンをまとめて見ます。

```

// 基本的な type エイリアス
type UserName = string;           // type で型に別名を付け
る。UserName は単に string の別名。
type Age = number;                // Age は number の別
名。意味のある名前を付けるとコードが読みやすい。
type IsActive = boolean;         // IsActive =
boolean。

// オブジェクト型
type User = {                     // type でオブジェクト
型を定義 (interface でもほぼ同じ)。
  id: number;
  name: string;
  email: string;
  age: number;
};

const user: User = {              // User 型として変数を
作る。
  id: 1,
  name: "田中太郎",
  email: "tanaka@example.com",
  age: 30,
};

// ユニオン型 (interface ではできない)
type Status = "active" | "inactive" | "pending"; // 文字列リテラルのユニ
オン。3つの値のいずれかに限定。
type Id = string | number;        // string か number
のどちらでも入る型。

const userStatus: Status = "active"; // "active" は許可され
た値なので OK。
const userId: Id = "user_123";      // string が許可されて
いるので OK。

// タプル型
type Coordinate = [number, number]; // 2要素タプル。[緯度,
経度] のように使う。
type NameAge = [string, number];    // [名前, 年齢] のタプ
ル。

const point: Coordinate = [35.6762, 139.6503]; // 東京の座標
代入。 // Coordinate 型として

// 関数型
type MathOperation = (a: number, b: number) => number; // 「(a: number, b:
number) => number」は関数型の表記。「引数2つ受け取って number を返す関数」を意味する。

```



```
const add: MathOperation = (a, b) => a + b; // MathOperation 型として add を実装。引数の型は型から推論される。
const subtract: MathOperation = (a, b) => a - b; // 同じ型から別の関数を作れる（型の再利用）。
```

▼ コードを1つずつ分解して解説

`type` は「オブジェクトの形」以外にも色々な型に名前を付けられるのが強みです。代表的なパターンを、1つずついていに見ていきましょう。

解説1: 基本型に別名を付ける

```
type UserName = string;
type Age = number;
type IsActive = boolean;
```

- `type 名前 = 型;` の形で、既存の型に自分で好きな名前（別名）を付けられます。
- `type UserName = string` は「`UserName` は `string` の別名」という意味で、中身はただの文字列です。ただ「`string`」と書くより `UserName` と書くほうが「これは利用者名なんだな」と意図が伝わります。
- このように、意味のある名前を付けるとコードが読みやすくなる、というのが `type` の使いどころの1つです。

用語: 型エイリアス (type alias) ... 既存の型に付ける「別名 (あだ名)」のこと。 `type` キーワードで作ります。

解説2: オブジェクト型に名前を付ける

```
type User = {
  id: number;
  name: string;
  email: string;
  age: number;
};

const user: User = {
  id: 1,
  name: "田中太郎",
  email: "tanaka@example.com",
  age: 30,
};
```

- `type` でも、`interface` と同じようにオブジェクトの形を定義できます。`{ }` の中に「キー名: 型」を並べる書き方は `interface` とほぼ同じです。

- 違いは書き方だけで、`type User = { ... }` の `=`（イコール）が付く点が `interface User { ... }` との見た目の差です。
- 一度 `User` 型を作れば、`const user: User = { ... }` のように使い回せます。やっていることは `interface` の例とまったく同じです。

解説3: ユニオン型に名前を付ける（`type` ならでは）

```
type Status = "active" | "inactive" | "pending";
type Id = string | number;

const userStatus: Status = "active";
const userId: Id = "user_123";
```

- `|`（パイプ）で型を並べると「A または B」というユニオン型になります。これは `interface` ではできず、`type` ならではの機能です。
- `type Status = "active" | "inactive" | "pending"` は「`Status` 型には、この3つの文字列のどれかしが入らない」という意味です。「`active`」（打ち間違い）のような値を入れると即エラーになります。
- `type Id = string | number` は「`Id` 型には文字列か数値のどちらかが入る」という意味です。

用語: ユニオン型 ... `|` で複数の型をつないだ「どれか1つ」を表す型。「`string` または `number`」のように、複数の候補を許したいときに使います。

解説4: タプル型・関数型に名前を付ける

```
type Coordinate = [number, number];
type NameAge = [string, number];

const point: Coordinate = [35.6762, 139.6503]; // 東京の座標

type MathOperation = (a: number, b: number) => number;

const add: MathOperation = (a, b) => a + b;
const subtract: MathOperation = (a, b) => a - b;
```

- `[number, number]` はタプル型（位置ごとに型と個数が固定された配列）です。`Coordinate` という名前を付けて「座標（緯度・経度）」を表しています。
- `(a: number, b: number) => number` は関数型で、「数値を2つ受け取って数値を返す関数」という形を表します。これに `MathOperation` という名前を付けています。
- 一度 `MathOperation` 型を作れば、`add` と `subtract` のように同じ型から複数の関数を作る（型を使い回せる）のが利点です。引数 `a`・`b` の型は型注釈から自動で推論されるので、関数側に書く必要はありません。

用語: 関数型 ... 「どんな引数を受け取り、何を返す関数か」を表す型。(引数: 型) => 戻り値の型 という形で書きます。

▼ このコードがやること（先に日本語で）: `type` ならではの、少し進んだ型の組み合わせ方を見ます。`&`（交差型）で「両方の性質を合わせ持つ型」を作ったり、条件によって型を変える「条件付き型」、既存の型を一括変換する「マップ型」を扱います。最初は難しく感じる部分なので、まずは `&` で型を合体できる、という点だけ押さえればOKです。

```

// 交差型 (Intersection Type)
type HasName = {                                     // 「name を持つ」だけ
    name: string;
};
// 表す型。

type HasAge = {                                     // 「age を持つ」だけ
    age: number;
};
// 表す型。

type Person = HasName & HasAge;                     // & は交差型
// (intersection) の記号。両方の性質を合わせ持つ型。「name と age を両方持つ」になる。
// Person は { name: string; age: number; } と同じ

const person: Person = {
    name: "田中",
    age: 30,
};

// 条件付き型 (Conditional Type)
type IsString<T> = T extends string ? "yes" : "no"; // T が string 型に
// 含まれるなら "yes"、そうでないなら "no" を返す型。T extends X は「T が X 型の部分型か」の判定。

type A = IsString<string>; // "yes"                 // T = string →
// "yes"。
type B = IsString<number>; // "no"                  // T = number →
// "no"。

// マップ型 (Mapped Type)
type Readonly<T> = {                                 // T の全プロパティを
    readonly [P in keyof T]: T[P];                 // [P in keyof T]
};                                                    // は「T のキーをひとつずつ P として取り出す」記法。T[P] はそのキーに対応する値の型。前に readonly を
// 付けるとすべて読み取り専用になる。

type ReadonlyUser = Readonly<User>;                // User の全プロパティ
// 全てが readonly になった型ができる。
// すべてのプロパティが readonly になる

```

4.3 interface vs type の使い分け

機能	interface	type
オブジェクトの形を定義	OK	OK
継承 (extends)	OK	交差型 (&) で代替
実装 (implements)	OK	OK
宣言のマージ (Declaration Merging)	OK	不可
ユニオン型	不可	OK
タプル型	不可	OK
プリミティブ型のエイリアス	不可	OK
マップ型	不可	OK
条件付き型	不可	OK
計算されたプロパティ	不可	OK

▼ このコードがやること（先に日本語で）： **interface** だけが持つ「宣言のマージ」という機能を見ます。同じ名前で **interface** を2回書くと、TypeScript が自動で1つに合体してくれます（ライブラリの型を後から拡張するときに使う）。一方 **type** で同名を再定義するとエラーになる、という違いも確認します。

```
// --- interface でしかできないこと: 宣言のマージ ---
interface Window {                                     // 同じ名前の
interface を複数書くと、TypeScript が自動でマージしてくれる。
  title: string;
}

interface Window {
  appVersion: number;
}

// 2つの宣言がマージされる
// Window = { title: string; appVersion: number; }      // 結果は両方のプロパ
// ティを持つ型。ライブラリの型を後から拡張する用途で使う。

// type では同じ名前で再定義するとエラーになる
type Animal = { name: string };                        // 1つ目の定義。
type Animal = { age: number }; // エラー: Duplicate identifier 'Animal' // 同じ名前で再定
// 義しようとするエラー。
```

▼このコードがやること（先に日本語で）：逆に `type` だけができることを並べて見ます。ユニオン型（「AまたはB」）、プリミティブ型への別名付け、タプル型、関数型、条件付き型——これらは `interface` では書けません。「オブジェクトの形以外も名前を付けたいときは `type` 」と覚えるための一覧です。

```
// --- type でしかできないこと ---

// ユニオン型
type Result = "success" | "error" | "loading"; // | で複数のリテラルを並べて型を作るのは type ならでは。

// プリミティブのエイリアス
type ID = string | number; // プリミティブ型に別名を付けるのも type のみ。

// タプル型
type Point = [number, number]; // タプル型のエイリアスも type で書く。

// 関数型
type Formatter = (input: string) => string; // 関数型のエイリアスも type で書く。

// 条件付き型
type NonNullable<T> = T extends null | undefined ? never : T; // 条件付き型も type 限定。T が null か undefined なら never、そうでなければ T 自体。
```

使い分けの指針:

- **interface を使う場面:** オブジェクトの形状を定義する場合（特にクラスが実装する場合や、ライブラリの型を拡張したい場合）
- **type を使う場面:** ユニオン型、タプル型、関数型、プリミティブ型エイリアスなど、interface では表現できない型を定義する場合

実務でのコツ: チーム内で統一するのが最も重要です。迷ったら `interface` をデフォルトで使い、`interface` で表現できない場合のみ `type` を使う という方針が一般的です。

4.4 書籍管理アプリで使う型の定義例

実際のアプリケーション開発を想定して、書籍管理アプリに必要な型を定義してみましょう。初心者でもわかるように、ほぼ全行に解説コメントを付けています。

▼ このコードがやること（先に日本語で）：これまで学んだ型の道具（ユニオン型・ **interface** ・継承・ジェネリクスなど）を総動員して、これから作る書籍管理アプリ全体で使う「データの形」をまとめて定義します。ここで型を一度作っておくと、アプリのあらゆる場所で同じ型を再利用でき、間違いを TypeScript が指摘してくれます。長いですが、1つずつコメントを追えば大丈夫です。

```
// =====
// 書籍管理アプリの型定義（詳細コメント版）
// =====
// このファイルはアプリ全体で使う「データの形」を集めた辞書のようなもの。
// ここで型を1度作っておくと、関数の引数や useState の中身など
// あらゆる場所で同じ型を再利用でき、間違いを TypeScript が指摘してくれる。
// =====

// -----
// (1) 「読書ステータス」の型
// -----
// 「ユニオン型」(| で区切って列挙)を使い、値を3つの文字列のどれかに限定する。
// こうすると status = "yomu" のような typo が即座にエラーになる。
type ReadingStatus = "want-to-read" | "reading" | "completed";
//                               ↑読みたい       ↑読書中       ↑読了

// -----
// (2) 「評価」の型 - 1~5 の数値リテラルだけを許す
// -----
// number だけだと 0 や 100 も入ってしまう。1~5 のいずれかに限定したいので、
// 数値リテラル型のユニオンで宣言する。
type Rating = 1 | 2 | 3 | 4 | 5;

// -----
// (3) カテゴリの型 - 列挙したいときの定石
// -----
// 各値の右側のコメントは「人間用のメモ」。実行時には消える。
type BookCategory =
  | "fiction"           // 小説
  | "non-fiction"      // ノンフィクション
  | "technology"       // 技術書
  | "business"         // ビジネス
  | "self-help"        // 自己啓発
  | "other";           // その他

// -----
// (4) 著者情報の型 - interface で「オブジェクトの形」を定義
// -----
// `?` を付けたプロパティは「あってもなくてもいい」（オプション）
interface Author {
  id: string;           // 著者のユニークID（必須）
  name: string;         // 著者名（必須）
  nationality?: string; // 国籍（省略可能。? が付く）
}
```



```
// -----
// (5) 書籍の基本情報
// -----
// `author` プロパティは前で定義した Author 型を再利用している。
// このように型を入れ子にできるのが TypeScript の便利なところ。
interface Book {
  id: string;           // 書籍のユニークID
  title: string;        // タイトル
  author: Author;       // 著者 (Author 型のオブジェクト)
  isbn: string;          // ISBN番号 (書籍の世界共通ID)
  category: BookCategory; // カテゴリ (前述の6種類のどれか)
  pages: number;        // 総ページ数
  publishedDate: string; // 出版日。ISO 8601 形式 "2025-01-15"
  coverImageUrl?: string; // 表紙画像URL (省略可能)
  description?: string;  // 説明文 (省略可能)
}

// -----
// (6) 読書記録の型 - Book を「継承」して項目を追加
// -----
// `extends Book` は「Book のすべての項目をそのまま受け継いだうえで、追加項目を持つ」
// という意味。継承を使うと「Book にこれだけ足したやつ」という意図が明確になる。
interface ReadingRecord extends Book {
  status: ReadingStatus; // 読書ステータス (必須)
  rating?: Rating;        // 評価 (読了後にのみ設定するためオプショナル)
  startDate?: string;     // 読み始めた日
  endDate?: string;       // 読み終わった日
  notes?: string;         // メモ
  currentPage?: number;   // 現在のページ (読書中のみ意味を持つ)
}

// -----
// (7) ユーティリティ型 Omit / Partial の活用
// -----
// 同じことを毎回手書きすると面倒なので、TypeScript の組み込みユーティリティを使う。

// `Omit<T, K>` = T から K プロパティを抜いた型を作る。
// 新規作成時は id がまだ存在しないので、Book から id を抜く。
type CreateBookInput = Omit<Book, "id">;
// 結果として「id 以外の全フィールドが必須」の型ができる。

// `Partial<T>` = T のすべてのプロパティを「オプショナル」にした型。
// 更新時は「変えたい項目だけ」送れば良いので、全項目を ? 付きにしたい。
type UpdateBookInput = Partial<Omit<Book, "id">>;
// 結果として「id 以外の全フィールドが省略可」の型ができる。
```

```
// -----
// (8) フィルター条件の型 - 検索・並び替えのオプション
// -----
// すべて省略可 (?) にして、ユーザーが指定したものだけ使うパターン。
interface BookFilter {
  status?: ReadingStatus; // ステータスで絞る
  category?: BookCategory; // カテゴリで絞る
  searchQuery?: string; // タイトルまたは著者名で検索
  sortBy?: "title" | "publishedDate" | "rating"; // 並び替え基準
  sortOrder?: "asc" | "desc"; // asc=昇順 (1→9, A→Z), desc=降順 (9→1, Z→A)
}

// -----
// (9) ジェネリクスを使った API レスポンス型
// -----
// `<T>` は「あとで決まる型のプレースホルダー」。
// 例えば `ApiResponse<Book>` と書けば T = Book になり、
// `ApiResponse<Author>` と書けば T = Author になる。
// 「成功・失敗の枠組みは同じだが、データの中身は呼び出し側で決まる」を表現するのに最適。
interface ApiResponse<T> {
  data: T; // 取得したデータ本体 (型は呼び出し側次第)
  success: boolean; // 成否
  message?: string; // メッセージ (省略可)
}

// -----
// (10) ページネーション付きレスポンス
// -----
// 一覧取得APIは「データ配列+ページ情報」をセットで返すのが定番。
// data: T[] の T[] は「T の配列」を意味する記法。
interface PaginatedResponse<T> {
  data: T[]; // 1ページ分のデータ配列
  total: number; // 全件数
  page: number; // 現在のページ番号 (1始まり)
  perPage: number; // 1ページあたりの件数
  totalPages: number; // 総ページ数
}

// ===== 使用例 =====

// 書籍の作成
const newBook: CreateBookInput = {
  title: "TypeScript実践ガイド",
  author: {
    id: "author_001",
    name: "山田太郎",
    nationality: "日本",
  },
},
```

```

    isbn: "978-4-XXXX-XXXX-X",
    category: "technology",
    pages: 400,
    publishedDate: "2025-06-01",
    description: "TypeScriptの基礎から実践まで学べる一冊",
  };

// 読書記録
const myReading: ReadingRecord = {
  id: "book_001",
  title: "TypeScript実践ガイド",
  author: {
    id: "author_001",
    name: "山田太郎",
  },
  isbn: "978-4-XXXX-XXXX-X",
  category: "technology",
  pages: 400,
  publishedDate: "2025-06-01",
  status: "reading",
  startDate: "2025-07-01",
  currentPage: 150,
  notes: "第5章まで読了。ジェネリクスの説明がわかりやすい。",
};

// フィルターを使った検索
const filter: BookFilter = {
  status: "reading",
  category: "technology",
  sortBy: "title",
  sortOrder: "asc",
};

// API レスポンスの型適用
const response: ApiResponse<ReadingRecord> = {
  data: myReading,
  success: true,
  message: "書籍情報を取得しました",
};

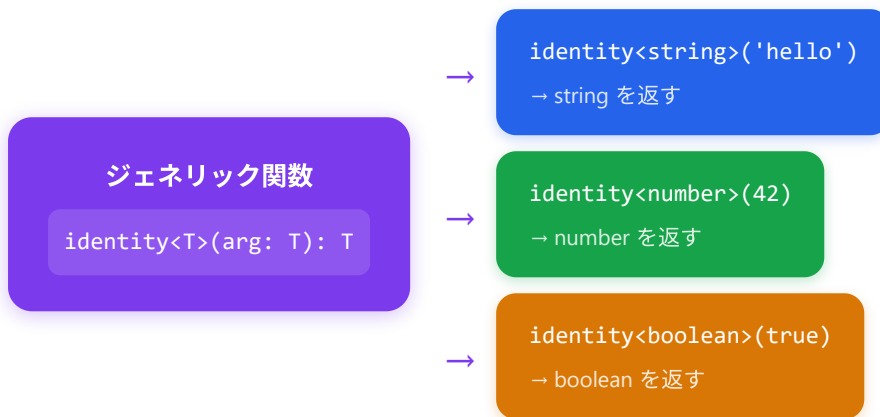
// ページネーション付きレスポンス
const listResponse: PaginatedResponse<ReadingRecord> = {
  data: [myReading],
  total: 1,
  page: 1,
  perPage: 10,
  totalPages: 1,
};

```

5. ジェネリクス

5.1 基本概念

ジェネリクス（Generics）は、型を「パラメータ化」する仕組みです。関数やクラス、インターフェースを定義する時点では型を決めず、使う時点で具体的な型を指定できます。



▼ このコードがやること（先に日本語で）：ジェネリクスのありがたみを実感するための「悪い例」を先に見ます。「受け取った値をそのまま返すだけ」の関数を作りたいのに、**any** を使うと型情報が失われ、型ごとに関数を作ると同じコードが量産されてしまう——という困りごとを示します。次のブロックでこれをジェネリクスが解決します。

```
// --- ジェネリクスなしの場合（問題あり） ---

// 方法1: any を使う → 型安全性が失われる
function identityAny(arg: any): any {                                // any を使えば何でも
  受け取れるが...
  return arg;
}
const result1 = identityAny("hello"); // result1 は any 型 (string 情報が失われる) // 結果も any 扱いになり、型安全性が消える。

// 方法2: 型ごとに関数を作る → コードの重複
function identityString(arg: string): string {                      // string 専用版。
  return arg;
}
function identityNumber(arg: number): number {                     // number 専用版。型が増えるごとに関数を量産する必要がある。
  return arg;
}
```

▼ このコードがやること（先に日本語で）：いよいよジェネリクスが登場です。<T> という「あとで決まる型の入れ物」を使い、1つの関数であらゆる型に対応させます。identity<string>(…) のように型を指定したり、値から自動推論させたりできます。カギは「T は型の変数（プレースホルダー）」という考え方。使うときに具体的な型が当てはまります。

```
// =====
// ジェネリクスを使った場合 - 「型を変数化」して1つの関数で全てに対応
// =====

// `` の T は「Type」の頭文字。慣習的によく使われる。
// 「この関数は何かの型 T を受け取り、同じ T を返すよ」という宣言。
//
// arg: T → 引数の型は T
// : T → 戻り値の型も T (引数と同じ型)
function identity<T>(arg: T): T {
  return arg; // 受け取ったものをそのまま返す
}

// ----- 使用例(1): 型を明示的に指定する -----
// `` のように山カッコで型を渡すことを「型引数を渡す」と呼ぶ。
const str = identity<string>("hello");
// ↑ T = string と決まるので、引数は string 限定、戻り値も string
console.log(str);
// ▼ 実行結果
// hello
// ▼ str の型: string
// (number を渡そうとするとコンパイルエラー: identity<string>(42) は ✖)

const num = identity<number>(42);
console.log(num);
// ▼ 実行結果
// 42
// ▼ num の型: number

const bool = identity<boolean>(true);
console.log(bool);
// ▼ 実行結果
// true
// ▼ bool の型: boolean

// ----- 使用例(2): 型を省略する (型推論にお任せ) -----
// 引数の値 "world" を見て、TypeScript が「あ、T = string ですね」と自動推論する。
// 多くの場合、型引数を書かなくてOK。
const inferred = identity("world");
console.log(inferred);
// ▼ 実行結果
// world
// ▼ inferred の型: string (自動推論された)
```

▼ コードを1つずつ分解して解説

ジェネリクスは初心者が最初にとまどう概念の代表です。<T> という見慣れない書き方を、1つずついねいに見ていきましょう。

解説1: ジェネリック関数の定義 (<T>)

```
function identity<T>(arg: T): T {  
    return arg; // 受け取ったものをそのまま返す  
}
```

- `function identity<T>(...)` の `<T>` がジェネリクスの正体です。これは「あとで決まる型を入れておく箱（型の変数）」で、関数を定義する時点では「何型か」を決めません。
- `arg: T` ... 引数 `arg` の型を `T` にしています。「`arg` の型は、あとで決まる `T` と同じ」という意味です。
- `: T` ... 戻り値の型も `T` にしています。つまり「受け取った型と同じ型を返す」関数だと宣言しています。
- `T` という名前は「Type（型）」の頭文字で、慣習でよく使われるだけです。`<U>` でも `<Item>` でも動きますが、まずは `T` に慣れましょう。

用語: ジェネリクス（総称型） ... 型を「あとで決める変数」のように扱う仕組み。`<T>` のように山カッコで書き、1つの関数や型をあらゆる型に対応させられます。

用語: 型パラメータ ... `<T>` の `T` のこと。関数の引数（パラメータ）が「値の入れ物」であるのに対し、型パラメータは「型の入れ物」です。

解説2: 使うとき型を明示的に指定する

```
const str = identity<string>("hello");  
const num = identity<number>(42);  
const bool = identity<boolean>(true);
```

- 関数名のうしろに `<string>` のように書くと、「今回の `T` は `string` ですよ」と型を渡せます。これを「型引数を渡す」と言います。
- `identity<string>("hello")` では `T = string` に決まるので、引数は文字列しか受け付けず、戻り値（`str`）の型も `string` になります。
- 同様に `identity<number>(42)` なら `T = number`、`identity<boolean>(true)` なら `T = boolean` になります。1つの関数が、渡した型に応じて姿を変えるのがジェネリクスの威力です。
- もし `identity<string>(42)` のように型と中身が食い違くと、TypeScript が即エラーで教えてくれます。

```
const inferred = identity("world");
// ▼ inferred の型: string (自動推論された)
```

- 多くの場合、`<string>` のような型引数は書かなくてもかまいません。
- `identity("world")` のように値だけ渡すと、TypeScript が引数 `"world"` を見て「あ、`T` は `string` だな」と自動で判断（型推論）してくれます。
- そのため `inferred` の型はちゃんと `string` になります。実務では、このように型を省いて書くことが多いです。

用語: 型推論 ... 型注釈を書かなくても、TypeScript が値から「これは何型か」を自動で判断してくれる機能のこと。

ジェネリクスのおかげで: 上の `identity` 関数は、もし型を毎回手書きだったら `identityString` `identityNumber` `identityBoolean` ... と無限に増えてしまう。`<T>` 1つで全てに対応できる。配列メソッドの `Array.prototype.map<U>(callback)` などはまさにこの仕組みで動いている。

▼ このコードがやること（先に日本語で）: 型パラメータは1つだけでなく、`<T, U>` のように複数持てます。ここでは2つの値をペア（タプル）にして返す `pair` 関数を作ります。あわせて `T · U · K · V` といった「型パラメータ名のよくある慣習」も紹介します（意味は普通の変数名と同じく自由ですが、慣習に従うと読みやすい）。

```
// --- ジェネリクスの慣習的な型パラメータ名 ---
// T: Type (一般的な型)
// U: 2番目の型パラメータ
// K: Key (オブジェクトのキー)
// V: Value (オブジェクトの値)
// E: Element (要素)
// R: Return (戻り値)

// 複数の型パラメータ
function pair<T, U>(first: T, second: U): [T, U] {           // <T, U> で2種類の型
  パラメータを宣言。タプル [T, U] を返す。                // 引数2つをタプルに
  return [first, second];                                   // して返す。
}

const p1 = pair<string, number>("hello", 42);             // [string, number] // 型を明示。
// T=string, U=number.
const p2 = pair("田中", true);                             // [string, boolean] (型推論) // 引数から自動推論。
```


5.2 実用的な例

例1: 配列操作のユーティリティ関数

▼このコードがやること（先に日本語で）：ジェネリクスの実用例として、「どんな型の配列でも使える」便利関数を3つ作ります——最初の要素を取る `getFirst`、最後の要素を取る `getLast`、中身をシャッフルする `shuffle` です。`<T>` のおかげで、文字列の配列でも数値の配列でも、**同じ関数1つで型安全に扱えます**。

```
// 配列の最初の要素を取得する関数
function getFirst<T>(arr: T[]): T | undefined { // <T> を導入。引数の T[] で「T の配列」を受け取る。戻り値は T かもしれない（空配列のとき）。
    return arr[0]; // [0] は1要素目。配列が空なら undefined になる。
}

const firstFruit = getFirst(["りんご", "みかん"]); // string | undefined // T = string と推論。
const firstNum = getFirst([10, 20, 30]); // number | undefined // T = number と推論。

// 配列の最後の要素を取得する関数
function getLast<T>(arr: T[]): T | undefined { // 同じく <T> を使う汎用関数。
    return arr.length > 0 ? arr[arr.length - 1] : undefined; // 三項演算子。長さか0より大きければ最後の要素、そうでなければ undefined。
}

// 配列をシャッフルする関数
function shuffle<T>(arr: T[]): T[] { // 配列を受け取り、シャッフルした新しい配列を返す。
    const result = [...arr]; // ... はスプレッド構文。「配列の中身を全部展開する」演算子。[...arr] で「元配列を1段コピーした新しい配列」を作れる。元を破壊しないため。
    for (let i = result.length - 1; i > 0; i--) { // 末尾から先頭手前まで逆順にループ (Fisher-Yates シャッフル)。
        const j = Math.floor(Math.random() * (i + 1)); // Math.random() は 0以上1未満の乱数。* (i+1) で 0..i の小数、Math.floor で切り捨てて整数に。
        [result[i], result[j]] = [result[j], result[i]]; // 分割代入による値の入れ替え。左右を同時に評価して交換する。
    }
    return result; // シャッフル後の配列を返す。
}

const shuffledFruits = shuffle(["りんご", "みかん", "バナナ"]); // string[]
const shuffledNums = shuffle([1, 2, 3, 4, 5]); // number[]
```

▼ コードを1つずつ分解して解説

3つの関数それぞれにジェネリクスの使いどころが詰まっています。順番に、1つずついねいに見ていきましょう。

解説1: 最初の要素を取る `getFirst`

```
function getFirst<T>(arr: T[]): T | undefined {  
    return arr[0];  
}  
  
const firstFruit = getFirst(["りんご", "みかん"]); // string | undefined  
const firstNum = getFirst([10, 20, 30]);           // number | undefined
```

- `getFirst<T>(arr: T[]) ... <T>` で型を変数化し、引数を `T[]` (`T` の配列) にしています。これで「どんな型の配列でも受け取れる」関数になります。
- 戻り値の型は `T | undefined` です。なぜ `undefined` が付くかというと、もし空の配列を渡されたら `arr[0]` (1番目の要素) が存在せず `undefined` になるからです。「無いかもしれない」を型で正直に表しています。
- `getFirst(["りんご", "みかん"])` では `T = string` と自動推論され、戻り値は `string | undefined` になります。数値の配列を渡せば `number | undefined` です。同じ関数1つで型ごとに正しく対応できるのがジェネリクスの利点です。

解説2: 最後の要素を取る `getLast`

```
function getLast<T>(arr: T[]): T | undefined {  
    return arr.length > 0 ? arr[arr.length - 1] : undefined;  
}
```

- こちらも `<T>` を使った汎用関数で、配列の最後の要素を返します。
- `arr.length > 0 ? ... : ...` は三項演算子です。「条件 ? 条件が真のときの値 : 偽のときの値」という形で、ここでは「要素が1個以上あるなら最後の要素、なければ `undefined`」という意味になります。
- `arr[arr.length - 1] ... arr.length` は要素数です。添え字は0始まりなので、最後の要素の添え字は「要素数 - 1」になります (例: 3個なら添え字は2)。

用語: 三項演算子 ... `条件 ? A : B` の形で「条件が真なら A、偽なら B」を返す式。短い条件分岐を1行で書けます。

```
function shuffle<T>(arr: T[]): T[] {
  const result = [...arr];
  for (let i = result.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [result[i], result[j]] = [result[j], result[i]];
  }
  return result;
}
```

- `const result = [...arr]` ... `...` (ドット3つ) はスプレッド構文で、「元の配列の中身を全部展開して、新しい配列にコピーする」書き方です。元の `arr` を壊さないために、コピーを作ってから加工しています。
- `for (let i = ...; i > 0; i--)` ... 末尾から先頭手前まで逆向きに繰り返すループです。これは「フィッシャー・イェーツ法」というシャッフルの定番アルゴリズムです。
- `Math.floor(Math.random() * (i + 1))` ... `Math.random()` は「0以上1未満の乱数」を返します。`*` (`i + 1`) で範囲を広げ、`Math.floor` (切り捨て) で整数にして、`0` から `i` までのランダムな添え字 `j` を作っています。
- `[result[i], result[j]] = [result[j], result[i]]` ... 分割代入を使った値の入れ替えです。一時変数を使わずに、`result[i]` と `result[j]` の中身を一度に交換しています。
- 戻り値の型は `T[]`。シャッフル後の新しい配列を返します。

用語: スプレッド構文 (`...`) ... 配列やオブジェクトの中身を「展開」する書き方。`[...arr]` で配列を1段コピーでき、元を壊さずに加工するのに使います。

例2: ジェネリックなインターフェース

▼このコードがやること (先に日本語で) : ジェネリクスは関数だけでなく `interface` にも使えます。ここでは「成功・失敗の枠組みは共通だが、中身のデータの型は使う側で決める」API レスポンス型 `ApiResponse<T>` を作ります。`ApiResponse<User>` ならデータは `User`、`ApiResponse<Book[]>` なら書籍の配列、というふうに型を差し替えられるのがポイントです。

```

// API レスポンスの汎用型
interface ApiResponse<T> {
  // ジェネリックなイン
  // data の中身の型
  data: T;
  // HTTP ステータス
  status: number;
  // Date は
  (200, 404, 500, ...)
  message: string;
  timestamp: Date;
  JavaScript 標準の日時オブジェクト。
}

// ユーザーデータ用のレスポンス
interface User {
  id: number;
  name: string;
  email: string;
}

const userResponse: ApiResponse<User> = {
  // T = User として
  ApiResponse を使う。data は User 型に決まる。
  data: {
    id: 1,
    name: "田中太郎",
    email: "tanaka@example.com",
  },
  status: 200,
  message: "成功",
  timestamp: new Date(),
  // new Date() は
  「今この瞬間の日時」を表す Date オブジェクトを作る。
};

// 書籍データ用のレスポンス
interface Book {
  id: string;
  title: string;
}

const bookResponse: ApiResponse<Book[]> = {
  // T = Book[] (書籍
  // の配列) として使う。data は Book[] に決まる。
  data: [
    { id: "1", title: "TypeScript入門" },
    { id: "2", title: "React実践ガイド" },
  ],
  status: 200,
  message: "成功",
  timestamp: new Date(),
};

```

例3: 制約付きジェネリクス (extends)

▼ このコードがやること（先に日本語で）：「どんな型でもOK」ではなく「ある条件を満たす型だけ受け取りたい」ときに使う、制約付きジェネリクス（`<T extends ...>`）を学びます。例えば「length を持つものだけ」「そのオブジェクトに実在するキーだけ」のように絞り込めます。少し難しい所なので、`extends` で「型に条件を付けられる」とつかめれば十分です。

```
// T は必ず length プロパティを持つ型に制限
function getLength<T extends { length: number }>(arg: T): number { // <T extends X>
  // 「T は X 型の部分型でなければならない」という制約。ここでは「length: number を持つ何か」に制限。
  return arg.length; // length プロパ
  // ティが必ずあるので安全に読める。
}

getLength("hello"); // OK: string は length を持つ → 5 // 文字列は length
// を持つ。
getLength([1, 2, 3]); // OK: 配列は length を持つ → 3 // 配列も length を
// 持つ。
getLength({ length: 10 }); // OK: length プロパティがある → 10 // 自前で { length:
// 10 } を渡しても OK。

// getLength(42);
// エラー: Argument of type 'number' is not assignable to
// parameter of type '{ length: number; }'
// → number には length プロパティがない // number に
// length は無い → 制約違反。

// オブジェクトのキーに制限をかける
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] { // K は「T のキーの
  // 中のいずれか」に制限。keyof T は T のキーのユニオン型を作る演算子。T[K] は「T 型オブジェクトの K プ
  // ロパティの型」を取り出す書き方。
  return obj[key];
}

const user = { name: "田中", age: 30, email: "tanaka@example.com" }; // ↑ user の型は {
// name: string; age: number; email: string; } と推論される。

const name = getProperty(user, "name"); // string // K="name" なの
// で戻り値の型は user.name の型 = string。
const age = getProperty(user, "age"); // number // K="age" なので
// number。

// getProperty(user, "address"); // "address" は
// user のキーに含まれない。
// エラー: Argument of type '"address"' is not assignable to
// parameter of type '"name" | "age" | "email"'
```

例4: ジェネリックなクラス

▼ このコードがやること（先に日本語で）：ジェネリクスをクラスに使う例として、「最後に入れたものが最初に出てくる」スタックというデータ構造を `Stack<T>` として作ります。`new Stack<number>()` なら数値専用、`new Stack<string>()` なら文字列専用、というふうに、同じクラスを違う型で何度でも再利用できるのがポイントです。

```

// スタック (Last In, First Out) のデータ構造
class Stack<T> { // class はオブ
ジェクトの設計図を定義するキーワード。<T> でジェネリッククラスにする。「T 型の要素を入れるスタッ
ク」。
    private items: T[] = []; // private = ク
ラスの外からはアクセス禁止。T[] 型の空配列で初期化。

    push(item: T): void { // メソッドの宣
言。スタックに要素を追加。
        this.items.push(item); // this は「こ
のインスタンス自身」を指す。items 配列に追加。
    }

    pop(): T | undefined { // スタックの末
尾を取り出して返す。空なら undefined。
        return this.items.pop(); // 配列の pop
メソッドは末尾を削除して返す（破壊的）。
    }

    peek(): T | undefined { // 末尾を「見る
だけ」（削除しない）。
        return this.items[this.items.length - 1]; // length-1
が最後の要素の添え字。
    }

    isEmpty(): boolean { // 空かどうか。
        return this.items.length === 0;
    }

    size(): number { // 要素数を返
す。
        return this.items.length;
    }
}

// 数値スタック
const numberStack = new Stack<number>(); // T = number
で Stack を作る。new はクラスからインスタンスを作るキーワード。
numberStack.push(10);
numberStack.push(20);
numberStack.push(30);
console.log(numberStack.pop()); // 30 // 最後に push
した 30 が取り出される (LIFO の性質)。
console.log(numberStack.peek()); // 20 // 次に末尾にあ
るのは 20。

// 文字列スタック
const stringStack = new Stack<string>(); // T = string
で別のスタックを作る。同じ Stack クラスを違う型で再利用できる。

```

```
stringStack.push("TypeScript");
stringStack.push("React");
console.log(stringStack.pop()); // "React"
```

6. ユニオン型とリテラル型

6.1 ユニオン型

ユニオン型（Union Type）は、複数の型のいずれかを受け入れる型です。`|`（パイプ）で型を区切ります。

▼ このコードがやること（先に日本語で）：「`string` または `number`」のように、複数の型のどれかを受け入れるユニオン型（`|` で区切る）を学びます。大事なのは「使うときは中身が今どっちの型かを `typeof` で確かめてから使う」こと——この確認を**型ガード**と呼びます。配列に対して使うときは `( )` の付け方で意味が変わる点にも注意します。


```
// =====
// ユニオン型の基本 - 「A型 または B型」を受け入れる型
// =====

// (1) string 型 または number 型 に入れられる変数を宣言する
//   let は const と違って後から再代入できる。
//   | は「または」の意味 (AND ではなく OR)。
let value: string | number;

value = "hello"; // OK (string なので許可)
value = 42;      // OK (number なので許可)
// value = true;
// ▼ エラー
// Type 'boolean' is not assignable to type 'string | number'.
//   (boolean は string|number に代入不可)

// =====
// 関数の引数にユニオン型を使う場合の「型の絞り込み」
// =====
// ユニオン型のままでは「両方の型に共通するメソッドしか」呼べない。
// typeof 演算子などで「いま中身が何の型か」を確かめてから使う。
// この技法を「型ガード (Type Guard)」と呼ぶ。

function formatId(id: string | number): string {
  // (1) typeof 演算子は「値の型を文字列で返す」演算子
  //   文字列なら "string"、数値なら "number" を返す。
  if (typeof id === "string") {
    // (2) この if ブロックの中だけ、TS は id を string として扱ってくれる。
    //   なので string 専用メソッド .toUpperCase() が使える。
    return id.toUpperCase(); // 例: "abc" → "ABC"
  } else {
    // (3) else 側では「string ではない」=「number しか残らない」と推論される。
    //   .toString() は数値を文字列に変換するメソッド。
    //   .padStart(5, "0") は「5文字に満たないなら左を 0 で埋める」メソッド。
    return id.toString().padStart(5, "0"); // 例: 42 → "00042"
  }
}

console.log(formatId("abc"));
// ▼ 実行結果
// ABC

console.log(formatId(42));
// ▼ 実行結果
// 00042

// =====
```

```
// 配列のユニオン型 - 配列の中に複数の型が混ざってもOKにする
// =====
// 注意: () で囲まないと意味が変わる!
// string | number[] → 「string 1個」または「number の配列」
// (string | number)[] → 「string と number が混在した配列」 (こちらが正しい)
const mixed: (string | number)[] = [1, "two", 3, "four"];
console.log(mixed);
// ▼ 実行結果
// [ 1, 'two', 3, 'four' ]
```

▼ コードを1つずつ分解して解説

ユニオン型は「型ガード」とセットで理解するのが大事です。順番に、1つずついねいに見ていきましょう。

解説1: ユニオン型で「A または B」を表す

```
let value: string | number;

value = "hello"; // OK (string なので許可)
value = 42;      // OK (number なので許可)
// value = true; // ▼ エラー
```

- `: string | number` がユニオン型の型注釈です。`|` (パイプ) は「または」を意味し、「`string` または `number` のどちらかが入る」という型になります。
- そのため `value = "hello"` (文字列) も `value = 42` (数値) もOKですが、`value = true` (真偽値) は許可されていない型なのでエラーになります。
- ここでは `let` を使っている点に注目してください。`let` は再代入できるので、文字列を入れたあとに数値を入れ直せます。

用語: ユニオン型 ... `|` で複数の型をつないだ「どれか1つ」を表す型。許したい型の候補が複数あるときに使います。

解説2: 型ガードで「いま中身が何型か」を確かめる

```
function formatId(id: string | number): string {
  if (typeof id === "string") {
    return id.toUpperCase(); // 例: "abc" → "ABC"
  } else {
    return id.toString().padStart(5, "0"); // 例: 42 → "00042"
  }
}
```

- 引数 `id` は `string | number` 型です。このままだと「文字列と数値の両方に共通する操作」しかできません（片方にしかないメソッドは呼べない）。
- そこで `if (typeof id === "string")` で「いま `id` は文字列か？」を確認しています。`typeof`（タイプオブ）は値の型を文字列で返す演算子です。
- この `if` の中では、TypeScript が「ここでは `id` は `string` で確定」と判断してくれるので、文字列専用の `.toUpperCase()`（大文字化）が安全に呼べます。
- `else` 側では「文字列ではない＝残るのは `number` だけ」と判断され、数値を文字列にする `.toString()` などを使えます。この「中身の型を絞り込んでから使う」技法を型ガードと呼びます。

用語: 型ガード (Type Guard) ... `typeof` などで「いま値が何型か」を確認し、その範囲内で型を絞り込むこと。ユニオン型を安全に使うための必須テクニックです。

解説3: 配列に使うときは `( )` の位置に注意

```
const mixed: (string | number)[] = [1, "two", 3, "four"];
```

- 「文字列と数値が混ざった配列」を作りたいときは、`(string | number)[]` と書きます。`( )` でユニオン型を囲んでから `[]` を付けるのがポイントです。
- もし `( )` を付けずに `string | number[]` と書くと、意味が変わってしまいます。これは「`string` 1個」または「`number` の配列」という別物になります。
- 「混在した配列」を表したいときは必ず `( )` で囲む、と覚えておきましょう。

6.2 リテラル型

リテラル型 (Literal Type) は、特定の値のみを許可する型です。

▼ このコードがやること（先に日本語で）: 「`string` なら何でも」ではなく、「`"north"`・`"south"` ... のように決められた値だけ」を許すリテラル型を学びます。`type Direction = "north" | "south" | ...` のようにユニオン型と組み合わせて使い、typo（打ち間違い）を即座にエラーにできます。文字列・数値・真偽値それぞれの例を見ます。

```
// =====
// 文字列リテラル型 - 「決められた文字列のどれか」だけを許す型
// =====
// type で名前を付け、許可する値を | で並べる。Enum の代わりによく使う。

type Direction = "north" | "south" | "east" | "west";
//           ↑ この4つの文字列以外は受け付けない

let dir: Direction = "north"; // OK (許可されている値)
// dir = "up";
// ▼ エラー
// Type '"up"' is not assignable to type 'Direction'.

console.log(dir);
// ▼ 実行結果
// north

// =====
// 数値リテラル型 - 「決められた数値のどれか」だけを許す型
// =====
// サイコロの目 (1~6) のように値が固定の場面で使う。
type DiceRoll = 1 | 2 | 3 | 4 | 5 | 6;

let roll: DiceRoll = 3; // OK
// roll = 7;
// ▼ エラー
// Type '7' is not assignable to type 'DiceRoll'.

// =====
// 真偽値リテラル型 - true だけ、または false だけを許す
// =====
// 滅多に使わないが、特定の値しか取らないフラグを表現したいとき便利。
type True = true;
let flag: True = true; // OK
// flag = false; // ▼ エラー: Type 'false' is not assignable to type 'true'.
```

6.3 書籍管理アプリでの実践例

▼ このコードがやること（先に日本語で）：リテラル型のユニオンを、実際の書籍管理アプリの「読書ステータス」に応用します。ReadingStatus（「読みたい・読書中・読了」の3択）を型として定義し、Book の status をその3つだけに限定します。**"finished"** のような決められていない値を入れると即エラーになる——これが型で「データの正しさ」を守る実例です。

```
// ===== 読書ステータスの定義 =====

type ReadingStatus = "want-to-read" | "reading" | "completed"; // 3つの文字列リテラルだけを許す型。

interface Book { // 書籍の基本型。
  id: string;
  title: string;
  author: string;
  status: ReadingStatus; // ステータスはReadingStatus 型に限定。
}

// --- 正しい使い方 ---
const book1: Book = {
  id: "1",
  title: "TypeScript入門",
  author: "山田太郎",
  status: "reading", // 許可された値。
};

const book2: Book = {
  id: "2",
  title: "React実践ガイド",
  author: "佐藤花子",
  status: "completed",
};

const book3: Book = {
  id: "3",
  title: "Next.js入門",
  author: "鈴木一郎",
  status: "want-to-read",
};

// --- エラーになる例 ---
const invalidBook: Book = {
  id: "4",
  title: "間違った書籍",
  author: "テスト太郎",
  status: "finished", // エラー! // "finished" はReadingStatus の3つに含まれていない。
  // Type '"finished"' is not assignable to type 'ReadingStatus'
  // → "want-to-read" | "reading" | "completed" のどれかにしてください
};
```

▼ このコードがやること（先に日本語で）：読書ステータスを使って、よくある実務処理——ラベルや色やアイコンへの変換、ステータスでの絞り込み（`filter`）、ステータスの変更——をまとめて作ります。`switch` で全パターンを書けば、TypeScript が「漏れなく処理しているか」まで見てくれます。ステータス変更では「元を壊さず新しいオブジェクトを作る（`{ ...book, status: ... }`）」鉄則も登場します。

```
// ===== ステータスに応じた処理 =====

function getStatusLabel(status: ReadingStatus): string { // ステータスを
    日本語ラベルに変換する関数。
    switch (status) { // switch 文で
        値ごとに分岐。
        case "want-to-read": // status が
            "want-to-read" のとき。
            return "読みたい"; // return する
            と関数が即終了。break 不要。
        case "reading":
            return "読書中";
        case "completed":
            return "読了";
        }
        // すべてのケースを処理しているため、default 不要 // ReadingStatus の
        3値すべてを書いたので、TypeScript は「漏れなし」と判断する。
        // TypeScript がすべてのケースが網羅されていることを保証
    }

function getStatusColor(status: ReadingStatus): string { // ステータス →
    色コード（#で始まる16進数）の変換。
    switch (status) {
        case "want-to-read":
            return "#f39c12"; // オレンジ
        case "reading":
            return "#3498db"; // 青
        case "completed":
            return "#27ae60"; // 緑
        }
    }

function getStatusIcon(status: ReadingStatus): string { // ステータス →
    アイコン名の変換（map 方式）。
    const icons: Record<ReadingStatus, string> = { // Record<K,
        V> は「K 型のキーを全部持ち、値がすべて V 型」のオブジェクトを表す Utility Type。
        "want-to-read": "bookmark", // キー名にハ
        イフンが含まれるのでクォートが必要。
        reading: "book-open", // 普通の識別
        子はクォート省略可。
        completed: "check-circle",
    };
    return icons[status]; // status を
    キーにして値を取り出す。Record で全キー網羅されているので必ず文字列が取れる。
}

// ===== フィルタリング =====

function filterBooksByStatus( // 書籍配列を
```

```

「指定ステータスのものだけ」に絞り込む関数。
    books: Book[], // 書籍の配列
を受け取る。
    status: ReadingStatus // 絞り込みたいステータス。
): Book[] { // 戻り値は
    Book の配列。
    return books.filter((book) => book.status === status); // filter
    で「book.status と引数 status が一致する要素」だけ残す。
}

const allBooks: Book[] = [book1, book2, book3]; // 3冊をまとめた配列。
const readingBooks = filterBooksByStatus(allBooks, "reading"); // "reading"
    だけ残す → [book1]。
const completedBooks = filterBooksByStatus(allBooks, "completed"); //
    "completed" だけ残す → [book2]。

// ===== ステータスの変更 =====

function updateBookStatus( // book に新しい status を当てた新しい book を返す関数。
    book: Book,
    newStatus: ReadingStatus
): Book {
    return { ...book, status: newStatus }; // ... はスプレッド構文（オブジェクト版）。book の全プロパティをコピーして、後ろの status を上書き。元の book は壊さない（イミュータブル）。
}

const updatedBook = updateBookStatus(book1, "completed"); // book1 の status だけ "completed" にした新オブジェクト。
console.log(updatedBook.status); // "completed"

// 無効なステータスへの変更はコンパイルエラー
// updateBookStatus(book1, "abandoned"); //
// ReadingStatus にない値なのでエラー。
// エラー: Argument of type '"abandoned"' is not assignable to type 'ReadingStatus'

```

▼このコードがやること（先に日本語で）：TypeScript の鉄板パターン「判別可能なユニオン型」を学びます。

status という共通の目印（タグ）を使い、「読みたい本は理由を持つ」「読書中の本は現在ページを持つ」のように、状態ごとに持つ情報を変える型を作ります。**switch (book.status)** で分岐すると、その枝の中だけそのバリエーション固有のプロパティに安全にアクセスできます。


```
// ===== 判別可能なユニオン型 (Discriminated Union) =====
// 判別可能なユニオン型 = 「共通のキー (タグ)」を使って、ユニオン型の中の
// どのバリエーションかを区別できる型。TypeScript で複雑な状態を扱う鉄板パターン。

// ステータスに応じて異なる追加情報を持つ型
type BookWithDetails =
  | { // ユニオンの
    1つ目: status="want-to-read" のとき。 // ↑ 「判別
      status: "want-to-read"; // タグ」。これがあるとTSがバリエーションを区別できる。
      title: string;
      reason: string; // 読みたい理由 // この種類だ
    } // 2つ目:
  | {
    status="reading" のとき。
      status: "reading";
      title: string;
      currentPage: number; // 読書中だ
    } // 3つ目:
  | {
    status="completed" のとき。
      status: "completed";
      title: string;
      rating: 1 | 2 | 3 | 4 | 5; // 数値リテ
      review?: string; // review
    } // はオプション。
  };

function displayBookInfo(book: BookWithDetails): string {
  switch (book.status) { //
    case "want-to-read":
      // ここでは book.reason にアクセス可能 // この case
      // 内では TS が「book は『1つ目』のバリエーション」と絞り込む。
      return `「${book.title}」を読みたい (理由: ${book.reason})`;

    case "reading":
      // ここでは book.currentPage, book.totalPages にアクセス可能 // この case
      // 内では「2つ目」に絞り込み。currentPage が読める。
      const progress = Math.round( //
        Math.round (book.currentPage / book.totalPages) * 100 // 進捗率
      );
      return `「${book.title}」を読書中 (進捗: ${progress}%)`;
  }
}
```

```

    case "completed":
        // ここでは book.rating, book.review にアクセス可能 // この case
        // では「3つ目」に絞り込み。
        const stars = "★".repeat(book.rating) + "☆".repeat(5 - book.rating); //
        // .repeat(n) は文字列を n 回繰り返すメソッド。「★★★★☆」のような星評価を作る。
        return `「${book.title}」读了 ${stars}`;
    }
}

// 使用例
console.log(
    displayBookInfo({
        status: "reading",
        title: "TypeScript入門",
        currentPage: 150,
        totalPages: 400,
    })
);
// => 「TypeScript入門」を読書中（進捗: 38%）

console.log(
    displayBookInfo({
        status: "completed",
        title: "React実践ガイド",
        rating: 4,
        review: "とても実践的でした",
    })
);
// => 「React実践ガイド」读了 ★★★★★

```

7. TypeScript の設定（tsconfig.json）

7.1 tsconfig.json とは

tsconfig.json は TypeScript プロジェクトの設定ファイルです。コンパイラのオプション、対象ファイル、出力先などを指定します。

```

{
  "compilerOptions": {
    "target": "ES2022",
    // TypeScript コンパイラ (tsc) に渡す設定。
    // コンパイル後の JavaScript のバージョン。新
    // モジュールシステム。"ESNext" = ES Modules
    "module": "ESNext",
    // 使える組み込み型の集合。"DOM" を入れるとブラ
    // ウザの window や document 等の型が使える。
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "strict": true,
    // 厳格モード一括有効化。これだけで厳しい型チェ
    // ックが7~8項目まとめてONになる。必須。
    "esModuleInterop": true,
    // CommonJS と ES Modules の互換性を良くす
    // る。 import の書き心地が良くなる。
    "skipLibCheck": true,
    // node_modules 内の型定義のチェックを省略し
    // てビルドを高速化。
    "forceConsistentCasingInFileNames": true,
    // ファイル名の大文字小文字を厳格にチェック。
    // macOS/Linux の差を吸収。
    "resolveJsonModule": true,
    // import data from "./foo.json" のように
    // JSON を読み込める。
    "isolatedModules": true,
    // 1ファイルだけ見て変換できる前提を強制。Babel
    // や SWC との互換性のため。
    "noEmit": true,
    // tsc から .js ファイルを出力しない。
    "jsx": "react-jsx",
    // JSX (React の <Component/> 構文) の変換
    // 方式。React 17+ の新形式。
    "baseUrl": ".",
    // パスエイリアスの基準ディレクトリ。"." はプ
    // ロジェクトルート。
    "paths": {
      "@/*": [".src/*"]
      // 短縮パスの定義。
      // "@/foo" と書けば ".src/foo" を指す。深
    },
    // いネストでも import が短く書ける。
  },
  "include": ["src/**/*"],
  // コンパイル対象。** は「任意の階層」、* は
  // 「任意のファイル」。
  "exclude": ["node_modules", "dist"]
  // 除外対象。ライブラリやビルド成果物はチェック
  // しない。
}

```

7.2 主要な設定項目

設定項目	説明	推奨値	備考
<code>target</code>	コンパイル先の JavaScript バージョン	<code>"ES2022"</code>	モダンブラウザ対象なら ES2022 以降
<code>module</code>	モジュールシステム	<code>"ESNext"</code>	ES Modules を使用
<code>lib</code>	使用する型定義ライブラリ	<code>["ES2022", "DOM"]</code>	ブラウザ用に DOM を含める
<code>strict</code>	すべての厳格チェックを有効化	<code>true</code>	必ず <code>true</code> にする
<code>noEmit</code>	JavaScript ファイルを出力しない	<code>true</code>	Next.js 等のバンドラーを使う場合
<code>jsx</code>	JSX の処理方法	<code>"react-jsx"</code>	React 17+ の新しい JSX Transform
<code>esModuleInterop</code>	CommonJS/ESModule の相互運用	<code>true</code>	import 構文の互換性向上
<code>skipLibCheck</code>	型定義ファイルのチェックを省略	<code>true</code>	コンパイル速度の向上
<code>forceConsistentCasingInFileNames</code>	ファイル名の大文字小文字を区別	<code>true</code>	OS 間の差異を防止
<code>resolveJsonModule</code>	JSON ファイルの import を許可	<code>true</code>	JSON をモジュールとして読み込める
<code>isolatedModules</code>	ファイル単位のトランスパイルを保証	<code>true</code>	Babel 等との互換性
<code>baseUrl</code>	モジュール解決のベースパス	<code>(".")</code>	パスエイリアスの基準
<code>paths</code>	パスエイリアスの定義	<code>{"@/*": ["./src/*"]}</code>	<code>@/components</code> のような短縮パス

strict モードに含まれる個別オプション

`"strict": true` を設定すると、以下のオプションがすべて有効になります。

オプション	説明
<code>strictNullChecks</code>	<code>null</code> / <code>undefined</code> を他の型と区別する
<code>strictFunctionTypes</code>	関数の引数の型チェックを厳密にする
<code>strictBindCallApply</code>	<code>bind</code> , <code>call</code> , <code>apply</code> の引数を厳密にチェック
<code>strictPropertyInitialization</code>	クラスプロパティの初期化を必須にする
<code>noImplicitAny</code>	暗黙の <code>any</code> 型を禁止する
<code>noImplicitThis</code>	暗黙の <code>this</code> 型を禁止する
<code>alwaysStrict</code>	JavaScript の strict mode を有効にする
<code>useUnknownInCatchVariables</code>	catch の変数を <code>unknown</code> 型にする

7.3 Next.js での TypeScript 設定

Next.js プロジェクトでは、`next.config.ts` と `tsconfig.json` の両方で TypeScript の設定を行います。Next.js が自動的に最適な `tsconfig.json` を生成してくれます。

```

// Next.js プロジェクトの tsconfig.json (自動生成される)
{
  "compilerOptions": {
    "target": "ES2017", // 古めのブラウザも考慮した出力バージョン。
    "lib": ["dom", "dom.iterable", "esnext"], // ブラウザの DOM 系 + 最新の JS 機能の型
    // を有効化。
    "allowJs": true, // .js ファイルも TypeScript と同居でき
    // るようにする。
    "skipLibCheck": true,
    "strict": true, // 厳格モード ON。
    "noEmit": true, // Next.js 側のバンドラ (SWC) が出力す
    // るので、tsc では出力しない。
    "esModuleInterop": true, // バンドラ向けの解決方式。最新の Next.js
    // 推奨。
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve", // JSX を変換せずそのまま残す。あとで
    // Next.js (SWC) が変換する。
    "incremental": true, // 前回の型情報を再利用してビルドを高速
    // 化。
    "plugins": [
      {
        "name": "next" // Next.js 用 TypeScript プラグインを
    // 有効化 (型補完が強化される)。
      }
    ],
    "paths": {
      "@/*": [".src/*"] // @/foo → ./src/foo のショートカッ
    // ト。
    }
  },
  "include": [
    "next-env.d.ts", // Next.js が自動生成する型定義ファイ
    // ル。
    "**/*.ts", // すべての .ts ファイル。
    "**/*.tsx", // JSX を含む .tsx ファイル。
    ".next/types/**/*.ts" // .next ビルド出力中の型ファイル。
  ],
  "exclude": ["node_modules"]
}

```

ポイント: Next.js では `"jsx": "preserve"` が使われます。これは JSX の変換を Next.js (SWC) に任せるためです。`"noEmit": true` なので TypeScript コンパイラ自身は JavaScript を出力しません。

▼ このコードがやること (先に日本語で) : これから使う Next.js で実際に登場する「型付き」コードを先取りで眺めます。ページの情報 (`Metadata`)、サーバーでデータを取得する `async` (非同期) 関数と `await`、`props` の型定義などです。今は細部を理解しなくてOK——「これまで学んだ型の知識が、こういう実際の画面コードで使われるんだ」と雰囲気をつかむのが目的です。

```
// Next.js 特有の型の例

// ページコンポーネントの型
import type { Metadata } from "next"; // import type は「型情報
// だけをインポートする」構文。コンパイル後には何も残らない。

export const metadata: Metadata = { // ページのメタ情報 (ブラウ
// ザのタブ表示やSEOで使われる)。
  title: "書籍管理アプリ",
  description: "あなたの読書を管理するアプリです",
};

export default function HomePage() { // export default = この
// ファイルの「主役」をエクスポート。
  return <h1>書籍管理アプリ</h1>; // JSX。HTML のように見え
// るが実は関数呼び出しの構文糖衣。
}

// Server Component のデータ取得
interface PageProps { // Next.js のページコン
// ポーネントに渡される props の型。
  params: Promise<{ id: string }>; // URL のパスパラメー
// タ。Next.js 15+ では Promise に包まれている。
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>; // URL のク
// エリ文字列。[key: string] は「任意の文字列キー」を表すインデックスシグネチャ。
}

export default async function BookPage({ params }: PageProps) { // async = 非同期関数。{
// params } は分割代入で props から params だけ取り出す。
  const { id } = await params; // await は Promise の
// 中身を取り出す演算子。params から id を分割代入。
  // サーバーサイドでデータを取得
  const book = await fetchBook(id); // fetchBook(id) も
// Promise を返すので await。
  return <div>{book.title}</div>; // JSX の中で { 式 }
// と書くと、その値を埋め込める。
}
```

▼ コードを1つずつ分解して解説

ここには次章以降で頻出する `import type` や `async` / `await` が登場します。初見でこわく見える部分を、1つずついねいに見ていきましょう。

解説1: 型だけをインポートする (`import type`)

```
import type { Metadata } from "next";

export const metadata: Metadata = {
  title: "書籍管理アプリ",
  description: "あなたの読書を管理するアプリです",
};
```

- `import type { Metadata } from "next" ... import type` は「型情報だけを取り込む」専用の構文です。`Metadata` という型を `next` というライブラリから借りてきています。
- `type` を付けると「これは型のためだけのインポートだ」と TypeScript に伝わり、コンパイル後の JavaScript には何も残りません（型は実行時に消えるため）。
- `export const metadata: Metadata = { ... } ... : Metadata` の型注釈で「この `metadata` は `Metadata` 型の形をしている」と宣言しています。`title` や `description` のスペルを間違えたり型を間違えたりすると、すぐエラーで気づけます。
- `export` は「このファイルの外から使えるように公開する」キーワードです。

用語: `import type ...` 「型だけ」を取り込むインポート。実行時のコードには影響しないため、ビルド後のファイルが軽くなる利点があります。

解説2: props の型を定義する (`Promise` 入り)

```
interface PageProps {
  params: Promise<{ id: string }>;
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>;
}
```

- `PageProps` は、Next.js のページ部品に渡される入力データ (props) の「形」を定義した型です。
- `params: Promise<{ id: string }> ... Promise<...>` は「いますぐではなく、少し後で値が用意される」ことを表す型です。`{ id: string }` が将来手に入る中身で、ここでは URL の `id` です。
- `searchParams` の `{ [key: string]: ... }` はインデックスシグネチャといい、「任意の文字列をキーに持てるオブジェクト」を表します。URL のクエリ文字列はどんなキーが来るか事前に決められないため、この書き方を 사용합니다。値が `string | string[] | undefined` のユニオン型なのは「1個・複数個・無し」のどれもあり得るからです。

- 今は細部を覚えなくてOK。「URL の情報が **Promise** に包まれて渡ってくるんだな」とだけつかめれば十分です。

用語: Promise (プロミス) ... 「いまはまだ無いが、あとで用意される値」を表す入れ物。サーバーへの問い合わせなど「時間のかかる処理」の結果を表すのに使います。

解説3: 非同期関数でデータを取得する (**async** / **await**)

```
export default async function BookPage({ params }: PageProps) {
  const { id } = await params;
  const book = await fetchBook(id);
  return <div>{book.title}</div>;
}
```

- **async function** ... 関数の前に付く **async** (エイシンク) は「この関数は非同期処理を含む」という印です。**async** を付けると、その中で **await** が使えるようになります。
- **({ params }: PageProps)** ... 引数で props を受け取り、その場で分割代入して **params** だけを取り出しています。**: PageProps** は引数の型注釈です。
- **const { id } = await params** ... **await** (アウエイト) は「**Promise** の中身が用意されるまで待って、その値を取り出す」演算子です。**params** は **Promise** なので **await** で中身を待ち、そこから **id** を分割代入で取り出しています。
- **const book = await fetchBook(id)** ... **fetchBook(id)** も **Promise** を返す (データ取得に時間がかかる) ので、**await** で結果を待ちます。
- **return <div>{book.title}</div>** ... 取得した **book** のタイトルを JSX に埋め込んで画面に返しています。**{ }** の中には JavaScript の式を書けます。

用語: async / await ... 「時間のかかる処理 (サーバー通信など)」を、まるで順番に上から実行するかのように書ける仕組み。**async** を付けた関数の中で **await** を使い、結果が出るまで待ってから次に進みます。

8. よくあるエラーと対処法

TypeScript を使い始めると、必ず遭遇するエラーがあります。ここでは代表的なエラーとその解決方法を詳しく解説します。

8.1 Type 'X' is not assignable to type 'Y'

最も頻出するエラー です。ある型の値を、互換性のない別の型に代入しようとした際に発生します。

▼このコードがやること（先に日本語で）：最頻出エラー「Type 'X' is not assignable to type 'Y」（X型はY型に代入できません）」の典型パターンと直し方を、悪い例→良い例の順で見ます。要は「箱の型と中身の型が合っていない」状態。文字列を数値に変換する、型注釈を直す、`as const` を使う、といった具体的な対処法を覚えるのが目的です。

```
// ===== エラーパターン1: プリミティブ型の不一致 =====

// 悪い例
const age: number = "25"; // number 型に "25" (文字列) を入れている → 型違反。
// エラー: Type 'string' is not assignable to type 'number'

// 良い例
const age: number = 25; // 数値リテラルを直接入れる。
// または、文字列から変換する場合
const age: number = parseInt("25", 10); // parseInt は文字列を整数に変換する関数。第2引数 10 は「10進数として解釈」の意味。
const age: number = Number("25"); // Number() は文字列・boolean などを number に変換する関数。
```

```
// ===== エラーパターン2: オブジェクト型の不一致 =====

interface User { // 型を定義。
  name: string;
  age: number;
}

// 悪い例
const user: User = {
  name: "田中",
  age: "30", // string を number に代入 // age は number でなければ
            // ならないのに文字列。
};
// エラー: Type 'string' is not assignable to type 'number'

// 良い例
const user: User = {
  name: "田中",
  age: 30, // 数値に修正。
};
```

// ===== エラーパターン3: ユニオン型のリテラルが一致しない =====

```
type Status = "active" | "inactive"; // 許可される値は "active"
// か "inactive" のみ。
```

// 悪い例

```
const status: Status = "enabled"; // "enabled" は含まれていな
い。
// エラー: Type '"enabled"' is not assignable to type 'Status'
```

// 良い例

```
const status: Status = "active";
```

// ===== エラーパターン4: 変数の型が広すぎる =====

```
type Color = "red" | "blue" | "green";
```

// 悪い例

```
let colorName = "red"; // string と推論される // let は再代入されうるので
「広い型」(string) に推論されてしまう。
const color: Color = colorName; // string は Color (3リテ
ラルだけ) に代入できない。
// エラー: Type 'string' is not assignable to type 'Color'
```

// 良い例 (方法1: as const を使う)

```
const colorName = "red" as const; // "red" リテラル型と推論 // as const は「値をできる限り
狭いリテラル型に固定する」キーワード。
const color: Color = colorName; // OK
```

// 良い例 (方法2: 型注釈を使う)

```
const colorName: Color = "red"; // 最初から Color 型とし
て宣言。
```

// 良い例 (方法3: satisfies を使う)

```
const colorName = "red" satisfies Color; // "red" リテラル型かつ Color として検証 //
satisfies は「この値は x 型を満たしますよね？と確認しつつ、値そのものの狭い型は維持する」演算子。as
より安全。
```

8.2 Object is possibly 'undefined'

`strictNullChecks` が有効な場合に発生する、`null` や `undefined` の可能性がある値にアクセスしようとしたときのエラーです。

▼このコードがやること（先に日本語で）：「Object is possibly 'undefined」（値が無いかもしれない）」エラーの対処法を、配列アクセスやオプションなプロパティなど色々な場面で見ます。直し方は3つ——① `if` で「無いか」を先に確認、② `?.`（無ければ `undefined` を返す）、③ `??`（無ければ代替値を使う）。クラッシュを防ぐ実践的なテクニック集です。

```
// ===== エラーパターン1: 配列の要素アクセス =====
```

```
const fruits = ["りんご", "みかん", "バナナ"];
```

```
// 悪い例
```

```
const first: string = fruits[0];
```

```
//
```

```
noUncheckedIndexedAccess が有効だと、配列[i] は string | undefined と推論される。
```

```
// エラー (strict 設定次第) : Type 'string | undefined' is not assignable to type 'string'
```

```
// 配列のインデックスアクセスは undefined を返す可能性がある
```

```
// 良い例 (方法1: undefined チェック)
```

```
const first = fruits[0];
```

```
// 型注釈を省くと
```

```
string | undefined。
```

```
if (first !== undefined) {
```

```
// undefined でないか
```

```
  確認する型ガード。
```

```
    console.log(first.toUpperCase()); // OK
```

```
// ガード内では string
```

```
  に絞り込まれている。
```

```
}
```

```
// 良い例 (方法2: デフォルト値)
```

```
const first = fruits[0] ?? "デフォルト";
```

```
// ?? で undefined の場
```

```
合のデフォルト値を用意。これで first は必ず string。
```

```
console.log(first.toUpperCase()); // OK
```

```
// ===== エラーパターン2: オプショナルプロパティ =====

interface User {
  name: string;
  email?: string; // オプショナル (string | undefined) // ? を付けると省略可能なプロパティになる。
}

const user: User = { name: "田中" }; // email を省略。値は undefined。

// 悪い例
console.log(user.email.toUpperCase()); // user.email は string|undefined。undefined に .toUpperCase は呼べない。
// エラー: Object is possibly 'undefined'
// email は設定されていないかもしれない

// 良い例 (方法1: if チェック)
if (user.email) { // truthy チェック
  // (undefined や "" でない)。
  console.log(user.email.toUpperCase()); // OK // ガード内では string に絞り込まれる。
}

// 良い例 (方法2: オプショナルチェイニング)
console.log(user.email?.toUpperCase()); // undefined なら undefined を返す // ?. を付けると「左が null/undefined なら、その場で undefined を返す」。クラッシュしない。

// 良い例 (方法3: null 合体演算子と組み合わせ)
console.log(user.email?.toUpperCase() ?? "メール未設定"); // 結果が undefined なら ?? の右側 "メール未設定" を使う。
```

// ===== エラーパターン3: Map.get() の戻り値 =====

```
const userMap = new Map<string, string>(); // Map は「キーと値のペア  
を保持するデータ構造」。<キーの型, 値の型> でジェネリック指定。  
userMap.set("user1", "田中"); // .set(キー, 値) で追  
加。
```

// 悪い例

```
const name: string = userMap.get("user1"); // .get(キー) は「キー  
が存在しないこともある」のを考慮して string | undefined を返す。  
// エラー: Type 'string | undefined' is not assignable to type 'string'  
// Map.get() は undefined を返す可能性がある
```

// 良い例

```
const name = userMap.get("user1"); // 型は string |  
undefined。  
if (name !== undefined) {  
  console.log(name); // OK  
}
```

// または

```
const name = userMap.get("user1") ?? "不明"; // 未設定キーに対するデフ  
ォルト値で string に確定。
```

```
// ===== エラーパターン4: document.querySelector =====

// 悪い例
const button = document.querySelector("#submit-btn"); // querySelector は
CSS セレクタで要素を探す DOM API。見つからないと null を返す。戻り値は Element | null。
button.addEventListener("click", handleClick); // null かもしれないも
のに addEventListener は呼べない。
// エラー: Object is possibly 'null'
// querySelector は要素が見つからない場合 null を返す

// 良い例 (方法1: null チェック)
const button = document.querySelector("#submit-btn");
if (button) { // null/undefined で
  // ない=要素ありを確認。
  button.addEventListener("click", handleClick); // ガード内では非
  null に絞り込み済み。
}

// 良い例 (方法2: 存在が確実な場合は Non-null assertion)
const button = document.querySelector("#submit-btn")!; // 末尾の ! は「非null
アサーション」。「絶対 null じゃないと俺が保証する」と TypeScript に伝える演算子。型からのみ null
を取り除く (実行時のチェックはしない)。
// ただし、要素が存在しない場合は実行時エラーになるので注意
button.addEventListener("click", handleClick);

// 良い例 (方法3: 型を絞込む)
const button = document.querySelector<HTMLButtonElement>("#submit-btn"); //
<HTMLButtonElement> で「ボタン要素を期待している」と型引数を渡す。戻り値は HTMLButtonElement |
null。
if (button instanceof HTMLButtonElement) { // instanceof は「特
  定のクラスのインスタンスか」を判定する演算子。
  button.disabled = true; // HTMLButtonElement のプロパティにアクセス可能 // ガード内では
  HTMLButtonElement に絞り込まれ、.disabled が使える。
}
```

8.3 Property does not exist on type

オブジェクトに存在しないプロパティにアクセスしようとした際に発生するエラーです。

▼ このコードがやること (先に日本語で): 「Property does not exist on type (そのプロパティは存在しません)」エラーの原因と直し方を見ます。多くは①プロパティ名の打ち間違い (TypeScript が正しい名前を提案してくれる)、②型定義に項目が足りない、③ユニオン型で「今どっちの型か」を絞り込めていない、のいずれか。順に対処法を確認します。

// ===== エラーパターン1: タイプミス =====

```
interface User {  
  name: string;  
  email: string;  
}
```

```
const user: User = { name: "田中", email: "tanaka@example.com" };
```

// 悪い例

```
console.log(user.emial);
```

// emial (タイプミス)。

User 型に emial プロパティは無い。

// エラー: Property 'emial' does not exist on type 'User'.

// Did you mean 'email'?

// TypeScript が「もしか

して email では？」とヒントもくれる。

// 良い例

```
console.log(user.email);
```

// ===== エラーパターン2: 型定義にプロパティが不足 =====

```
interface Product {  
  name: string;  
  price: number;  
}
```

```
const product: Product = { name: "ペン", price: 150 };
```

// 悪い例

```
console.log(product.description);
```

// Product 型に

description は存在しない。

// エラー: Property 'description' does not exist on type 'Product'

// 良い例 (方法1: 型定義を修正)

```
interface Product {
```

```
  name: string;
```

```
  price: number;
```

```
  description?: string; // プロパティを追加
```

// オプションル (?) で追加

すれば既存データを壊さない。

```
}
```

// 良い例 (方法2: オブジェクトリテラルにプロパティを追加する場合)

```
const productWithDesc = { ...product, description: "赤いボールペン" }; // スプレッドで既存  
プロパティを展開し、後ろに description を追加した新オブジェクトを作る。型は自動推論される。
```


// ===== エラーパターン3: ユニオン型での共通でないプロパティ =====

```
interface Dog {  
  kind: "dog";  
  bark(): void;  
}  
// 判別タグ。"dog" リテラル型。  
// bark メソッド (戻り値なし)。
```

```
interface Cat {  
  kind: "cat";  
  meow(): void;  
}
```

```
type Animal = Dog | Cat; // Dog または Cat。
```

// 悪い例

```
function makeSound(animal: Animal) {  
  animal.bark();  
}  
// Cat に bark は無い。  
ユニオン型では「全バリエーションに共通する性質」しか使えない。  
// エラー: Property 'bark' does not exist on type 'Animal'  
// Property 'bark' does not exist on type 'Cat'  
// → Animal が Cat の場合、bark() は存在しない
```

// 良い例 (型の絞り込み)

```
function makeSound(animal: Animal) {  
  if (animal.kind === "dog") {  
    animal.bark();  
  } else {  
    animal.meow();  
  }  
}  
// kind を使ってどちらか判別。  
// OK: Dog 型として認識  
// ガード内では Dog に絞り込まれる。  
// OK: Cat 型として認識  
// else 側では Cat に絞り込まれる。
```

```

// ===== エラーパターン4: API レスポンスの型が不足 =====

// 悪い例: JSON.parse の結果は any ではなく unknown として扱うべき
async function fetchData() { // async/await を使った非同期関数。
    const response = await fetch("/api/data"); // fetch でサーバーに
    HTTP リクエスト。await で結果待ち。
    const data = await response.json(); // any 型 // .json() の戻り値の型は
    デフォルトで any。型情報が欠落。

    // この時点では data の構造が不明
    console.log(data.items.length); // 何のチェックも無しに
    data.items.length にアクセス。
    // ← 実行時エラーの可能性あり
}

// 良い例: 型を明示的に定義
interface ApiData { // 返ってくる JSON の構造を型として定義。
    items: string[];
    total: number;
}

async function fetchData(): Promise<ApiData> { // 戻り値の型を
    Promise<ApiData> と明示。
    const response = await fetch("/api/data");
    const data: ApiData = await response.json(); // 取得結果を ApiData 型
    として扱う (型アサーション) 。

    // 型安全にアクセスできる
    console.log(data.items.length); // items は string[] と
    分かるので length が読める。
    console.log(data.total);

    return data;
}

```

8.4 エラー対処のまとめフローチャート



発展: アプリでは使っていない重要なTypeScript機能

この章の本編とアプリ作りでは登場しなかったけれど、**実務やほかの人のコードを読むとき必ず出てくるTypeScriptの機能をまとめて紹介します**。どれも「これ単体で動く小さな例」なので、気になったものから順に読んでもらって大丈夫です。難しく感じたら「こういう機能があるんだな」と眺めるだけでもOKです。

補足:「ユニオン型」「リテラル型」は本編の第6章ですでに扱ったので、ここでは省略します。

Utility Types（型を組み立て直す便利な工具箱）

▼このコードがやること（先に日本語で）：すでにある型をもとに「一部だけ省略可能にした型」「特定の項目だけ抜き出した型」などを**自動で作り出す**便利機能（Utility Types）を一通り使います。同じ型を何度も手書きしなくて済むのが利点で、ここでは **Partial**・**Pick**・**Omit**・**Required**・**Record** の5つをまとめて見ます。

```
// =====
// Utility Types のサンプル - 既存の型から新しい型を「自動生成」する道具
// =====

// もとになる型を1つ用意する。これを使い回していく。
interface User {
  id: number;      // ユーザーID (必須)
  name: string;    // 名前 (必須)
  email: string;   // メールアドレス (必須)
}

// (1) Partial<T> ... すべてのプロパティを「省略してもよい (?付き)」にした型を作る
//     → { id?: number; name?: string; email?: string } と同じ意味になる
const draft: Partial<User> = { name: "田中" }; // id も email も省略できる

// (2) Pick<T, キー> ... T から指定したキーだけ「抜き出した」型を作る
//     → { id: number; name: string } と同じ意味
const summary: Pick<User, "id" | "name"> = { id: 1, name: "田中" };

// (3) Omit<T, キー> ... T から指定したキーを「取り除いた」型を作る (Pick の逆)
//     → { id: number; name: string } (email を除いた形)
const noEmail: Omit<User, "email"> = { id: 1, name: "田中" };

// (4) Required<T> ... すべてのプロパティを「必須」にした型を作る (Partial の逆)
interface Config {
  host?: string;    // ? が付いていて省略可能
  port?: number;
}
const fullConfig: Required<Config> = { host: "localhost", port: 8080 }; // 両方とも必須になる

// (5) Record<キーの型, 値の型> ... 「キーと値の組」をまとめて表す型を作る
//     ここでは「文字列キー → number 値」の辞書を表している
const scores: Record<string, number> = {
  math: 80,
  english: 95,
};

console.log(draft, summary, noEmail, fullConfig, scores);
// ▼ 実行結果
// { name: '田中' } { id: 1, name: '田中' } { id: 1, name: '田中' } { host: 'localhost',
port: 8080 } { math: 80, english: 95 }
```

▼ コードを1つずつ分解して解説

Utility Types は名前と役割さえ覚えれば、あとは組み合わせるだけです。1つずつ見ていきましょう。

解説1: Partial（全部を「省略可能」にする）

```
const draft: Partial<User> = { name: "田中" };
```

- `Partial<User>` は、`User` 型のすべてのプロパティに `?`（省略可能マーク）を付けた型を自動で作ります。
- もとの `User` は `id`・`name`・`email` がすべて必須ですが、`Partial<User>` ではどれを書いても・書かなくても OK になります。
- 「入力途中の下書きデータ」や「一部だけ更新したいとき」によく使います。

用語: Utility Types（ユーティリティ型） ... TypeScript に最初から用意されている「型を加工する道具」のこと。
型の名前<もとの型> の形で使います。

解説2: Pick と Omit（必要な項目を選ぶ／いらない項目を捨てる）

```
const summary: Pick<User, "id" | "name"> = { id: 1, name: "田中" };  
const noEmail: Omit<User, "email"> = { id: 1, name: "田中" };
```

- `Pick<User, "id" | "name">` は、`User` から `id` と `name` だけを抜き出した型です。`"id" | "name"` のように `|` で複数のキーを並べられます。
- `Omit<User, "email">` は、`User` から `email` を取り除いた型です。`Pick` のちょうど逆です。
- 「この場面では一部のプロパティしか使わない」というとき、もとの型を壊さずに新しい型を作れます。

解説3: Required（全部を「必須」にする）

```
const fullConfig: Required<Config> = { host: "localhost", port: 8080 };
```

- `Required<Config>` は、`Config` の `?`（省略可能マーク）をすべて外して必須にした型です。`Partial` の逆の動きをします。
- もとの `Config` は `host?` と `port?` がどちらも省略可能ですが、`Required<Config>` では両方とも必ず書かないとエラーになります。

解説4: Record（キーと値の組をまとめて表す）

```
const scores: Record<string, number> = {  
  math: 80,  
  english: 95,  
};
```

- `Record<string, number>` は「キーが `string`、値が `number` の組がいくつでも入る」型です。
- `< >` の中は左がキーの型、右が値の型です。今回は「科目名（文字列）→ 点数（数値）」の辞書を表しています。
- 「どんなキー名が来るか事前に決まっていない辞書」を型で表したいときに便利です。

enum（列挙型 — 決まった選択肢に名前を付ける）

▼ このコードがやること（先に日本語で）：「赤・黄・青」のような決まった数の選択肢に、それぞれ分かりやすい名前を付けてまとめる仕組み（enum）を使います。`Signal.Red` のように名前で書けるので、コードを読んだときに何を表しているか一目で分かるのが利点です。

```
// =====
// enum のサンプル - 「決まった選択肢の集まり」に名前を付ける
// =====

// enum (イーナム) = 「列挙型」。関連する定数 (変わらない値) に名前を付けてまとめる。
enum Signal {
  Red,      // 名前を書いただけだと、自動で 0 が割り当てられる
  Yellow,   // 次は 1
  Green,    // 次は 2
}

// enum の値は「列挙名.メンバー名」で取り出す
const current: Signal = Signal.Red;    // current には 0 が入る (が、意味は「赤信号」)

// 値そのものではなく「名前」で比較できるので読みやすい
if (current === Signal.Red) {
  console.log("止まれ");
}

// 数値ではなく文字列を割り当てることもできる (こちらの方がデバッグ時に分かりやすい)
enum Direction {
  Up = "UP",
  Down = "DOWN",
  Left = "LEFT",
  Right = "RIGHT",
}

const move: Direction = Direction.Up;  // move には "UP" が入る
console.log(move);

// ▼ 実行結果
// 止まれ
// UP
```

▼ コードを1つずつ分解して解説

enum は「選択肢に名前を付ける」だけのシンプルな機能です。1つずつ見ていきましょう。

解説1: 数値が自動で割り当たる enum

```
enum Signal {  
  Red,  
  Yellow,  
  Green,  
}
```

- `enum`（イナム）は「関連する選択肢をひとまとめにして名前を付ける」仕組みです。
- 値を書かずに名前だけ並べると、上から順に `0, 1, 2, ...` という数値が自動で割り当てられます。`Red` は `0`、`Yellow` は `1`、`Green` は `2` です。
- コードの中では `Signal.Red` のように「名前」で書けるので、`0` と直接書くより何を表しているか分かりやすくなります。

用語: enum（列挙型） ... 「とりうる値があらかじめ決まっている」ものに名前を付けてまとめた型。曜日・信号・状態など「種類が固定の選択肢」を表すのに向いています。

解説2: 文字列を割り当てる enum

```
enum Direction {  
  Up = "UP",  
  Down = "DOWN",  
  Left = "LEFT",  
  Right = "RIGHT",  
}  
  
const move: Direction = Direction.Up;
```

- `= "UP"` のように、各メンバーに好きな値（ここでは文字列）を指定することもできます。
- 数値の enum だと表示したとき `0` などになって意味が分かりにくいですが、文字列を割り当てておくと `console.log(move)` で `"UP"` と表示され、動作確認のとき分かりやすいという利点があります。

型ガード（今この値はどの型か、を絞り込む）

▼ このコードがやること（先に日本語で）：「文字列かもしれないし数値かもしれない」という値を扱うとき、**今この瞬間はどちらの型なのかを調べて安全に処理するテクニック（型ガード）**を学びます。**typeof**・**in**・自作の判定関数の3通りを使い、それぞれ「ここから先はこの型だ」とTypeScriptに伝える方法を見ます。


```
// =====
// 型ガードのサンプル - 「今この値はどの型か」を調べて安全に絞り込む
// =====

// (1) typeof による型ガード ... 文字列・数値などの「基本の型」を調べる
function describe(value: string | number): string { // value は string か number のどちらか
    if (typeof value === "string") {
        // この { } の中では value は string だと TypeScript が分かっている
        return `文字列です: ${value.toUpperCase()}`; // string 専用の .toUpperCase() が使える
    } else {
        // ここでは value は number に絞り込まれている
        return `数値です: ${value.toFixed(1)}`; // number 専用の .toFixed() が使える
    }
}

// (2) in による型ガード ... 「そのプロパティを持っているか」で型を見分ける
interface Dog {
    bark: () => void; // 犬は bark を持つ
}
interface Cat {
    meow: () => void; // 猫は meow を持つ
}

function speak(animal: Dog | Cat): void {
    if ("bark" in animal) { // animal が bark プロパティを持っているか調べる
        animal.bark(); // ここでは Dog に絞り込まれている
    } else {
        animal.meow(); // ここでは Cat に絞り込まれている
    }
}

// (3) 型述語 (x is T) による自作の型ガード関数
// 戻り値の型に「value is string」と書くと、true のとき value は string だと TS に伝わる
function isString(value: unknown): value is string {
    return typeof value === "string";
}

const data: unknown = "hello";
if (isString(data)) {
    // isString が true なので、ここでは data は string 扱い
    console.log(data.length); // string の .length が使える
}

console.log(describe("hello"));
console.log(describe(3.14159));
// ▼ 実行結果
```

```
// 文字列です: HELLO
// 数値です: 3.1
```

▼ コードを1つずつ分解して解説

型ガードは「if文で調べると、その中では型が確定する」のがポイントです。1つずつ見ていきましょう。

解説1: `typeof` で基本の型を見分ける

```
if (typeof value === "string") {
  return `文字列です: ${value.toUpperCase()}`;
} else {
  return `数値です: ${value.toFixed(1)}`;
}
```

- `typeof value` は「その値が何の型か」を文字列で返す演算子です（`"string"`、`"number"`、`"boolean"` など）。
- `if (typeof value === "string")` が成り立った `{ }` の中では、TypeScript が「ここでは `value` は `string` だ」と理解してくれます。だから `string` 専用の `.toUpperCase()` を安心して呼べます。
- `else` 側では残った可能性（`number`）に絞り込まれるので、`.toFixed()` が使えます。

用語: 型ガード ... 「今この値はどの型か」を調べて、コードの一部分だけ型を確定させる仕組みのこと。 `if` と組み合わせて使います。

解説2: `in` でプロパティの有無を見分ける

```
if ("bark" in animal) {
  animal.bark();
} else {
  animal.meow();
}
```

- `"プロパティ名" in オブジェクト` は「そのオブジェクトがそのプロパティを持っているか」を調べる書き方です。
- `Dog` だけが `bark` を持つので、`"bark" in animal` が成り立てば `animal` は `Dog` だと絞り込めます。
- 基本の型（`string/number` など）ではなく、**オブジェクトの種類を見分けたいときに便利**です。

```
function isString(value: unknown): value is string {  
    return typeof value === "string";  
}
```

- 関数の戻り値の型に `value is string` と書くと、その関数が `true` を返したとき「`value` は `string` だ」と TypeScript に教えられます。これを型述語と呼びます。
- 中身は `true` / `false` を返すだけの普通の関数ですが、`: boolean` の代わりに `: value is string` と書くのがポイントです。
- これを使うと `if (isString(data)) { ... }` の中で `data` を `string` として扱えるようになります。複雑な判定を関数にまとめて使い回せて便利です。

用語: `unknown` ... 「何が入っているか分からない値」を表す型。安全のため、型ガードで絞り込まないとほとんどの操作ができません。

as const (値を「変わらない決め打ちの値」として固定する)

▼ このコードがやること (先に日本語で) : 普通にかくと「ただの文字列」と扱われる値を、`as const` を付けることで「まさにその値だけ」という厳密な型に固定します。設定値や選択肢の一覧など「あとから変えたくない・特定の値だけ許したい」ものを安全に扱えるようになります。

```
// =====
// as const のサンプル - 値を「その値そのもの」に固定する
// =====

// (1) as const を付けない場合
let role1 = "admin"; // 型は string ("admin" 以外の文字列も入りうる、と判断される)

// (2) as const を付けた場合
const role2 = "admin" as const; // 型は "admin" (まさに "admin" という値だけ、に固定される)

// (3) オブジェクトに付けると、中身すべてが「変更不可&値固定」になる
const config = {
  host: "localhost",
  port: 8080,
} as const;
// config.port = 3000; // ✗ as const のため変更不可 (readonly になる)

// (4) 配列に付けると「読み取り専用&中身の値が固定されたタプル」になる
// よく「決まった選択肢の一覧」を作るのに使う
const colors = ["red", "green", "blue"] as const;
// colors の型は readonly ["red", "green", "blue"]

// この配列から「"red" | "green" | "blue"」というユニオン型を取り出せる
type Color = typeof colors[number]; // "red" | "green" | "blue"

const myColor: Color = "red"; // OK
// const ng: Color = "purple"; // ✗ 一覧に無い値はエラー

console.log(role2, config, colors, myColor);
// ▼ 実行結果
// admin { host: 'localhost', port: 8080 } [ 'red', 'green', 'blue' ] red
```

▼ コードを1つずつ分解して解説

`as const` は「値をガチガチに固定する」だけですが、便利な使い道があります。1つずつ見ていきましょう。

解説1: 付けるかどうかで型が変わる

```
let role1 = "admin"; // 型は string
const role2 = "admin" as const; // 型は "admin"
```

- `let role1 = "admin"` の型はただの `string` です。「文字列なら何でも入る箱」と判断されます。
- `"admin" as const` と書くと、型が `"admin"` という値そのものになります。つまり「`"admin"` 以外は入れられない」という厳密な型です。

- 「決まった値だけを許したい」ときに役立ちます。

用語: `as const` ... 値のうしろに付けると「この値は変わらない決め打ちの値だ」とTypeScriptに伝える書き方。型がぐっと厳密になります。

解説2: オブジェクト・配列に付けると全体が固定される

```
const config = {
  host: "localhost",
  port: 8080,
} as const;

const colors = ["red", "green", "blue"] as const;
```

- オブジェクトに `as const` を付けると、中のすべてのプロパティが `readonly`（変更不可）になり、値も固定されます。`config.port = 3000` のような書き換えはエラーになります。
- 配列に付けると「読み取り専用で、中身の値も順番も固定された配列（タプル）」になります。
- 「設定値」や「選択肢の一覧」のように、**あとから変えてほしくないデータ**にぴったりです。

解説3: 一覧からユニオン型を取り出す

```
const colors = ["red", "green", "blue"] as const;
type Color = typeof colors[number]; // "red" | "green" | "blue"
```

- `typeof colors` で「`colors` 変数の型」を取り出し、さらに `[number]`（配列の中身の型を取り出す書き方）を付けると、`"red" | "green" | "blue"` というユニオン型になります。
- こうすると「選択肢の一覧」と「許される値の型」を**1か所で管理**でき、片方を直せばもう片方も自動で揃います。選択肢が増えても書き換えるのは配列だけで済みます。

オプションルチェイニング `?.` と Null合体演算子 `??`

▼ このコードがやること（先に日本語で）: 「値がないかもしれない」場面で安全に値を取り出す2つの便利演算子を学びます。`?.` は「途中が空っぽならそこで止めてエラーを防ぐ」、`??` は「値が空っぽだったらこっちの値を使う」という働きです。データが欠けていてもプログラムを止めずに済みます。

```
// =====
// ?. と ?? のサンプル - 「値がないかも」を安全に扱う
// =====

interface User {
  name: string;
  address?: {           // address は省略可能 (無いかもしれない)
    city?: string;      // city も省略可能
  };
}

// (1) オプショナルチェイニング ?. ... 途中が null / undefined なら、そこで止めて undefined を返す
const user1: User = { name: "田中" };           // address が無い
console.log(user1.address?.city);               // address が無いので undefined (エラーにならない！)

const user2: User = { name: "鈴木", address: { city: "東京" } };
console.log(user2.address?.city);               // → "東京"

// もし ?. を使わずに user1.address.city と書くと、実行時にクラッシュしてしまう

// (2) Null合体演算子 ?? ... 左が null / undefined のときだけ、右の値を使う
const city1 = user1.address?.city ?? "未設定"; // 左が undefined なので "未設定"
const city2 = user2.address?.city ?? "未設定"; // 左が "東京" なのでそのまま "東京"

console.log(city1, city2);
// ▼ 実行結果
// undefined
// 東京
// 未設定 東京
```

▼ コードを1つずつ分解して解説

この2つはセットで使うことが多い演算子です。1つずつ見ていきましょう。

解説1: オプショナルチェイニング `?.`

```
const user1: User = { name: "田中" };
console.log(user1.address?.city); // undefined (エラーにならない)
```

- `?.` (オプショナルチェイニング) は「`?.` の左側が `null` か `undefined` なら、その場で止めて `undefined` を返す」演算子です。

- `user1.address` が無い (`undefined`) 場合、`user1.address.city` と普通に書くと「 `undefined` の中の `city` 」を読もうとして実行時にクラッシュします。
- `user1.address?.city` と書けば、途中で安全に止まって `undefined` が返るだけなので、プログラムが落ちません。

用語: オプショナルチェイニング ... `?.` を使って「途中が空っぽでも安全につなげて値を取り出す」書き方。

解説2: Null合体演算子 `??`

```
const city1 = user1.address?.city ?? "未設定";
```

- `??` (ヌル合体演算子) は「左側が `null` か `undefined` のときだけ、右側の値を使う」演算子です。
- `user1.address?.city` は `undefined` なので、`??` の右側の `"未設定"` が `city1` に入ります。
- 似た `||` (OR) と違い、`??` は `0` や空文字 `""` を「値あり」として扱う点が重要です。「`0` も正しい値」という場面では `??` を使うと安全です。

用語: Null合体演算子 ... `??` を使って「値が無いときだけ別の値 (デフォルト値) を使う」書き方。

ジェネリック関数 (型を後から決められる関数)

▼ このコードがやること (先に日本語で) : 「どんな型の値が来ても対応できる」関数の書き方 (ジェネリック関数) を学びます。 `<T>` という印を付けると「型はあとで呼び出すときに決める」という意味になり、文字列でも数値でも同じ1つの関数で扱えます。しかも型の情報は失われないので安全です。

```
// =====
// ジェネリック関数のサンプル - 「型をあとから決める」関数
// =====

// <T> が「型のプレースホルダ（あとで決まる型の仮の名前）」
// 引数 x の型も T、戻り値の型も「T の配列 (T[])」にしている。
function wrap<T>(x: T): T[] {
    return [x];          // 受け取った値を、要素1つだけの配列に包んで返す
}

// 呼び出すときに T が自動で決まる（明示しなくてよい）
const a = wrap("hello");    // T は string → a の型は string[]
const b = wrap(42);         // T は number → b の型は number[]
const c = wrap(true);       // T は boolean → c の型は boolean[]

console.log(a, b, c);
// ▼ 実行結果
// [ 'hello' ] [ 42 ] [ true ]

// 型を明示的に指定することもできる（普通は不要）
const d = wrap<string>("world"); // T を string と明示

console.log(d);
// ▼ 実行結果
// [ 'world' ]
```

▼ コードを1つずつ分解して解説

ジェネリックは「型に名前を付けて使い回す」だけの仕組みです。1つずつ見ていきましょう。

解説1: **<T>** は「あとで決まる型の名前」

```
function wrap<T>(x: T): T[] {
    return [x];
}
```

- 関数名のうしろの **<T>** は「型のプレースホルダ（仮の名前）」です。**T** は「あとで決まる型」を表す目印で、**T** という名前自体に決まりはありません（**<U>** でも **<Item>** でもOKです）。
- 引数 **x: T** は「**T** 型の値を受け取る」、戻り値 **: T[]** は「**T** の配列を返す」という意味です。受け取った型と返す型が連動しているのがポイントです。
- 中身の **return [x]** は「受け取った値を、要素1つだけの配列に包んで返す」だけのシンプルな処理です。

用語: ジェネリック（総称型） ... 型を「あとで決められる変数」のように扱う仕組み。1つの関数を色々な型で使い回せるようになります。

解説2: 呼び出すと型が自動で決まる

```
const a = wrap("hello"); // a の型は string[]
const b = wrap(42);      // b の型は number[]
```

- `wrap("hello")` のように文字列を渡すと、`T` が自動で `string` に決まり、戻り値 `a` の型は `string[]` になります。
- `wrap(42)` なら `T` は `number` になり、`b` は `number[]` です。渡した値から型が自動で決まるので、いちいち型を書く必要はありません。
- もし `any`（何でも型）で書いてしまうと型の情報が失われますが、ジェネリックなら「文字列を渡したら文字列の配列が返る」という対応が型として保たれるのが大きな利点です。

解説3: 型を明示的に指定することもできる

```
const d = wrap<string>("world"); // T を string と明示
```

- `wrap<string>(...)` のように `< >` の中に型を書くと、`T` を自分で指定できます。
- 普段は値から自動で決まるので省略しますが、「どうしてもこの型として扱ってほしい」場面では明示できる、と覚えておけば十分です。

まとめ

この章では、TypeScript の基礎として以下の内容を学びました。

トピック	学んだこと
TypeScript とは	JavaScript のスーパーセットであり、型安全性を提供する
基本的な型	<code>string</code> , <code>number</code> , <code>boolean</code> , <code>array</code> , <code>tuple</code> , <code>object</code> , <code>any</code> , <code>unknown</code> , <code>never</code> , <code>void</code> , <code>null</code> , <code>undefined</code>
型注釈と型推論	明示的に型を書く方法と、TypeScript が自動で型を判断する仕組み
interface と type	オブジェクトの形を定義する2つの方法とその使い分け
ジェネリクス	型をパラメータ化して再利用性を高める仕組み
ユニオン型とリテラル型	複数の型や特定の値のみを許可する型定義
tsconfig.json	TypeScript プロジェクトの設定ファイル
よくあるエラー	代表的な型エラーの原因と対処法

次の章へ

次の章では、**React の基礎**を学び、TypeScript と React を組み合わせた開発方法を習得していきます。この章で学んだ型の知識は、React コンポーネントの Props や State の定義で活用されます。